# 化工园区信息化平台 · 微服务架构设计方案（总文档）

> **技术栈**：Java / Spring Boot 3.x + Spring Cloud
> **版本**：V1.0 | **日期**：2026-08-22
> **说明**：本总文档整合 9 份子方案（架构设计、接口契约、事务实现、时序库、K8s 部署、等保三级、CI/CD、应急演练、ADR），用于技术方案统一存档与评审。

---

# 一、微服务架构设计方案

> **技术栈**：Java / Spring Boot 3.x + Spring Cloud **版本**：V1.0 | **日期**：2026-08-22 **依据**：原型 V1.0（10 模块 46 界面）、BRD / MRD / PRD / 产品路线图、应急管理部建设指南、GB/T 39218-2020、等保 2.0 三级 **设计理念**：安全为底线、数据为驱动、智能为方向；支撑"平急结合"、分期交付（M1–M36）、私有化 + 可选 SaaS

---

## 目录


1. 系统总体架构
2. 微服务模块划分
3. 服务间通信机制
4. API 网关设计
5. 服务注册与发现
6. 配置中心
7. 数据存储方案
8. 部署架构
9. 安全策略
10. 分期交付映射


---

## 一、系统总体架构

平台采用分层微服务架构，自顶向下分为五层：

> ```
> ┌──────────────────────────────────────────────────────────────────┐
> │  接入层  Web 门户 / 大屏(IOC) / 移动端(App·小程序) / 企业端 / 监管端        │
> ```

```
├──────────────────────────────────────────────────────────────┤
│  网关与安全层  Spring Cloud Gateway + OAuth2.1/OIDC + WAF + 限流熔断      │
├──────────────────────────────────────────────────────────────┤
│  应用微服务层  按业务域拆分（安环、应急、设备、人员定位、能碳、运营等）      │
├──────────────────────────────────────────────────────────────┤
│  共享支撑层  告警引擎 / 规则引擎 / 工作流 / 视频AI网关 / GIS空间服务 /       │
│              通知服务(短信·语音·广播·PUSH) / 文件服务 / 融合通信           │
├──────────────────────────────────────────────────────────────┤
│  数据中台层  采集接入 → Kafka → 实时计算(Flink) → 时序库/数仓 → 主数据     │
├──────────────────────────────────────────────────────────────┤
│  基础设施层  K8s / Nacos / Sentinel / Prometheus / ELK / Seata / MinIO   │
└──────────────────────────────────────────────────────────────┘
```

**设计原则**

- 每个微服务**独立数据库、独立部署、单一职责**（Database-per-Service）。
- 高频监测与应急指挥链路独立隔离，避免被日常流量拖垮。
- 聚合展示（IOC 大屏）作为 CQRS 读侧，不持有业务真相源。

---

## 二、微服务模块划分

按业务域驱动设计（DDD）拆分为 14 个服务，各服务私有数据相互独立。

### 2.1 服务清单

| # | 服务名 | 对应原型模块 | 核心职责 | 私有数据 |
|---|---|---|---|---|
| 1 | `gateway-svc` | 全局 | 路由、鉴权、限流、灰度发布 | 路由配置 |
| 2 | `iam-svc`（统一身份） | G4 | 用户/角色/三级权限(RBAC)、认证、审计日志 | 用户、角色、权限、操作日志 |
| 3 | `mdm-svc`（主数据） | G1 | 园区、企业、重大危险源、传感器、监测点主数据；数据资产目录 | 主数据、元数据 |
| 4 | `iot-ingest-svc`（采集接入） | 数据流入口 | 协议适配 OPC UA/Modbus/MQTT/HJ212/GB28181；边缘网关对接、数据清洗入 Kafka | 设备接入配置、点位映射 |
| 5 | `device-svc`（设备管理） | B2·E1–E4 | 设备/传感器全生命周期、资产台账、维保、在线状态 | 设备档案、维保记录 |
| 6 | `safety-svc`（安环管理） | B1·B4·B5·B6 | 一企一档、双重预防(风险清单+隐患排查)、特殊作业票、安全基础管理、环保台账联动 | 风险点、隐患、作业票 |

| # | 服务名 | 对应原型模块 | 核心职责 | 私有数据 |
|---|---|---|---|---|
| 7 | `monitor-svc`（重大危险源监测） | B2·B3 | 实时参数看板、四级风险预警、阈值规则执行 | 预警记录、阈值规则 |
| 8 | `closedmgmt-svc`（封闭化/人员定位） | B7·B8·B9 | 出入园门禁卡口、人员定位(UWB)、危化品车辆 GPS 追踪、电子围栏 | 出入记录、定位、围栏 |
| 9 | `env-svc`（环保监管） | C1–C5 | 环境质量、污染源 HJ212、VOCs、水环境、固废联单 | 监测点、排口、联单 |
| 10 | `energy-svc`（能碳管理） | D1–D4 | 能源计量、碳核算(Scope1/2/3)、能效诊断、源网荷储 | 能耗、碳配额 |
| 11 | `corridor-svc`（管廊设施） | E1–E4 | 管廊监控、泄漏检测(TDLAS/光纤)、无人机机器人巡检 | 管线、机库、巡检任务 |
| 12 | `emergency-svc`（应急指挥） | B10–B13·A4 | 预案库、资源一张图、指挥调度、事故模拟推演(ALOHA/PHAST/FDS) | 预案、资源、事件、推演 |
| 13 | `operation-svc`（园区运营） | F1–F5·H | 企业服务门户、智慧物流、产业链协同、OA、经济运行、移动端聚合 | 门户、物流、产业链、公文 |
| 14 | `ioc-svc`（智能运营中心） | A1–A3 | 态势一张图聚合、四色图、告警中心聚合（只读聚合服务） | 聚合视图缓存、告警聚合 |

## 2.2 核心模块职责边界说明

- **安环管理（`safety-svc` + `env-svc`）**：`safety-svc` 负责安全合规（风险、隐患、作业票、一企一档）；`env-svc` 负责环保全过程监管（排口、VOCs、固废联单）。两者共享主数据与设备，但数据真相源隔离。
- **应急指挥（`emergency-svc`）**：平急结合指挥中枢，独立高可用部署单元。日常维护预案与资源，应急时接管调度，对其他服务仅做**降级容错调用**（超时即走本地预案基线），保障"黄金十分钟"调度不依赖外部可用性。
- **设备管理（`device-svc`）**：统一管理传感/视频/定位/门禁/无人机资产，提供设备健康度给 monitor/env/corridor 复用，避免各域重复建设备表。
- **人员定位（`closedmgmt-svc`）**：集成 UWB/蓝牙/GPS 定位，承载出入园管控、人员实时位置、电子围栏越界、SOS 求助，并向应急指挥提供实时位置共享。
- **重大危险源监测（`monitor-svc`）**：独立于安全台账，单独库 + 读写分离，10s 刷新（PRD 性能要求），不被低频查询拖累。
- **智能运营中心（`ioc-svc`）**：CQRS 读模型，订阅事件 + 主动拉取组装大屏，不反向写业务，保证 30s 刷新与 ≤5s 加载。

---

# 三、服务间通信机制

## 3.1 通信方式选型

| 场景 | 方式 | 说明 |
| --- | --- | --- |
| 实时请求/响应（页面查询） | **OpenFeign（同步 REST）** | 经 Gateway，带链路追踪 |
| 跨域事件通知（预警、告警、事件） | **Kafka 异步事件** | 解耦、削峰、可重放 |
| 实时流式数据（IoT 点位） | **Kafka + WebSocket 推送** | iot-ingest → monitor/env/energy/device |
| 大屏聚合 | **ioc-svc 主动拉取 + 订阅告警事件** | CQRS 读模型 |

## 3.2 关键调用链

```
① 设备数据：企业DCS/PLC → iot-ingest-svc → Kafka{topic: iot.raw}
        → monitor-svc / env-svc / energy-svc / device-svc(状态)

② 预警闭环：monitor-svc 超阈值 → AlarmEvent(Kafka)
        → 告警引擎(分级) → notification-svc(推送)
        → ioc-svc(聚合展示) / emergency-svc(达应急等级) / 移动端(H2 PUSH)

③ 应急联动：closedmgmt-svc(车辆越界/SOS) → EmergencyTriggeredEvent
        → emergency-svc(接处警+资源推荐) → notification-svc(多通道调度)
        → closedmgmt-svc(位置共享) / ioc-svc(大屏切应急模式)

④ 作业票联动：safety-svc(特殊作业提交) → 调 device-svc(气体传感器)
        + monitor-svc(实时值) + 视频AI网关(现场画面) 联合抽查

⑤ 封闭化联动安全：closedmgmt-svc 黑名单拦截 → AlarmEvent → notification-svc + 安保
```

## 3.3 调用约束

- **禁止循环调用**：业务服务 → `ioc-svc` 只能"拉"（ioc 不反向写业务）。
- **应急优先**：`emergency-svc` 跨服务调用均带熔断降级与本地缓存兜底。
- **跨域只读**：`operation-svc` / `ioc-svc` 跨域查询走 Feign + Sentinel 熔断，失败返回兜底数据。

# 四、API 网关设计

采用 **Spring Cloud Gateway** 作为统一入口。

### 4.1 核心职责

- **统一路由**：按路径/服务名路由至后端微服务（如 `/api/safety/**` → `safety-svc`）。
- **统一鉴权**：校验 OAuth2.1 Access Token，解析身份与权限范围（模块/功能/数据级）。
- **限流熔断**：基于 Sentinel 的网关流控（按 IP/用户/API 维度），防止恶意刷量或雪崩。
- **灰度发布**：基于请求头/用户标签的权重路由，支持小流量验证新版本。
- **日志审计**：记录入口请求（脱敏），对接 ELK。
- **协议转换**：对外暴露 HTTPS/REST，内部可转发 gRPC（AI 推理等）。

### 4.2 网关路由示例

| 路由前缀 | 目标服务 | 鉴权 | 限流 |
| --- | --- | --- | --- |
| `/api/iam/**` | iam-svc | 部分公开(登录) | 中 |
| `/api/safety/**` | safety-svc | 必需 | 中 |
| `/api/monitor/**` | monitor-svc | 必需 | 高(实时) |
| `/api/emergency/**` | emergency-svc | 必需 | 高(应急优先) |
| `/api/closed/**` | closedmgmt-svc | 必需 | 中 |
| `/api/env/**` | env-svc | 必需 | 中 |
| `/api/energy/**` | energy-svc | 必需 | 低 |
| `/api/operation/**` | operation-svc | 必需 | 中 |
| `/api/ioc/**` | ioc-svc | 必需 | 高(大屏) |

### 4.3 容错

- 网关层超时 < 后端服务超时；后端不可用时返回标准化错误与兜底提示。
- 紧急态下 `emergency-svc` 路由标记 **最高优先级**，资源倾斜。

---

## 五、服务注册与发现

采用 **Nacos** 作为注册中心（同时承担配置中心，见第六节）。

### 5.1 机制

- 所有微服务启动时向 Nacos 注册实例（IP + 端口 + 元数据：环境/版本/健康权重）。
- 客户端（OpenFeign + LoadBalancer）从 Nacos 拉取服务列表，配合**本地缓存 + 健康检查**实现故障自动剔除。
- 支持**临时实例**（心跳保活）与**持久实例**（防止注册中心抖动导致大规模下线）。

### 5.2 高可用

- Nacos 集群部署（≥3 节点），Raft 一致性协议。
- 注册中心不可用时，服务依赖本地缓存继续运行（降级），不影响已建立的调用。

---

## 六、配置中心

采用 **Nacos Config** 统一管理配置。

### 6.1 配置组织

- **按环境隔离**：`dev` / `test` / `prod` 命名空间（Namespace）。
- **按服务分组**：每个微服务独立 Group（`safety-svc`、`monitor-svc` …）。
- **配置格式**：YAML，支持动态刷新（`@RefreshScope`）。

### 6.2 典型配置项

- 数据源、Kafka 地址、Redis 地址、Sentinel 规则。
- 业务参数：告警阈值、通知方式、数据刷新频率（对应 PRD G4 系统参数配置）。
- 等保相关：TLS 开关、审计级别、密码策略。

### 6.3 敏感配置

- 数据库密码、第三方 Key 经 Nacos 加密插件（AES）存储，禁止明文入库。
- 配置变更留痕，支持版本回滚。

---

## 七、数据存储方案

按"业务库 + 时序库 + 缓存 + 消息 + 对象存储"多引擎组合。

### 7.1 存储分层

| 数据类型 | 存储引擎 | 用途 |
| --- | --- | --- |
| 业务关系数据（企业/隐患/作业票/预案） | **PostgreSQL 14** | 主业务库，每服务独立 Schema/库 |
| 时序数据（IoT 点位、能耗、环保监测） | **TimescaleDB**（PostgreSQL 扩展） | 高频写入、时间窗口聚合、历史回溯 |
| 缓存/会话/热点 | **Redis 7** | 会话、设备在线状态、大屏聚合缓存 |
| 消息/事件流 | **Kafka 3.x** | 解耦、削峰、事件溯源 |
| 文件/视频/图片 | **MinIO**（对象存储） | 抓拍图、巡检报告、预案附件、视频片段 |

| 数据类型 | 存储引擎 | 用途 |
| --- | --- | --- |
| 日志 | **Elasticsearch**（ELK） | 操作审计、应用日志检索 |

### 7.2 多租户与隔离

- 单园区私有化部署：按 **Schema 隔离**（每园区/企业独立 Schema + 行级数据权限）。
- SaaS 多园区：按 **数据库实例隔离 + 行级过滤**（JDBC Filter 自动注入 tenant_id）。

### 7.3 数据一致性

| 数据类别 | 一致性要求 | 方案 |
| --- | --- | --- |
| 主数据(园区/企业/危险源) | 强一致、低频变更 | mdm-svc 单一真相源，其他域订阅快照到本地读模型 |
| IoT 实时点位 | 最终一致、可重传 | Kafka 持久化 + 至少一次；时序库幂等写入 |
| 告警/预警/应急事件 | 不丢、有序 | Kafka 分区键(设备ID/事件ID)保序 + 落库确认 |
| 隐患/作业票/审批 | 强一致 + 审计 | 单服务本地事务；跨服务用 **Seata Saga/TCC** |
| 跨域统计(IOC/经济运行) | 最终一致 | 事件溯源 + 定时核对 |

### 7.4 分布式事务（Seata）

- **Saga 模式**：长流程（特殊作业审批→气体检测→视频抽查→批准），每步有补偿。
- **TCC 模式**：资源类（应急资源调度锁定：预留→确认/取消）。
- **幂等**：所有 Kafka 消费者按 `eventId` 幂等；IoT 点位带 `timestamp+deviceNo` 去重。
- **状态机**：隐患/作业票流转用 Spring Statemachine 防非法跃迁。

### 7.5 备份策略

- 每日全量备份 + 实时增量（WAL 归档）；备份加密存储于对象存储异地副本。
- 故障恢复时间目标（RTO）≤ 30min，RPO ≤ 5min。

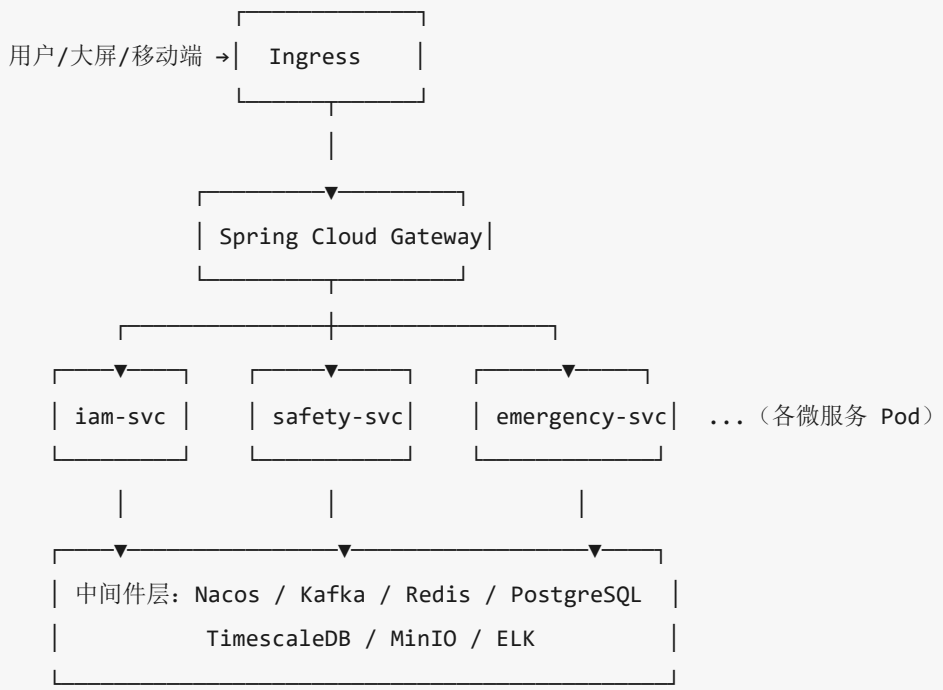
---

## 八、部署架构

### 8.1 容器化与编排

- 所有微服务打包为 **Docker 镜像**，由 **Kubernetes** 编排部署。
- 使用 **Helm Chart** 统一管理发布，支持按园区一键部署。
- 关键服务（monitor / emergency / ioc）配置 **Pod 反亲和 + 多副本 + HPA 自动扩缩容**。

### 8.2 环境拓扑




### 8.3 高可用与可靠性

- K8s 多节点集群，控制面与工作负载分离。
- 中间件均集群化：Nacos(≥3)、Kafka(≥3 broker + 副本)、PostgreSQL(流复制/Patroni)、Redis(哨兵/集群)。
- 7×24h 运行，年意外宕机 ≤ 3 次，操作失败率 < 0.5%。

### 8.4 边缘层

- 园区现场部署**边缘网关**（iot-ingest 轻量版），断网期间本地缓存数据，恢复后补传 Kafka，保障采集不丢。

---

## 九、安全策略

满足 **GB/T 22239-2019 等保 2.0 三级**要求。

| 控制项 | 实现方案 |
| --- | --- |
| 传输加密 | Gateway 全链路 **TLS 1.3**；内部服务间 mTLS |
| 存储加密 | 敏感字段 **AES-256**（证照、位置、视频元数据）；数据库透明加密(TDE) |
| 身份认证 | `iam-svc`：**OAuth2.1 + OIDC**；移动端 PUSH 令牌绑定设备 |
| 访问控制 | **三级 RBAC**：模块级 / 功能级 / 数据级（按企业 ID 行级隔离，多租户） |
| 审计留痕 | 操作日志**只追加(WORM)**，不可删除；ELK 归档，记录操作人/时间/IP/内容 |

| 控制项 | 实现方案 |
| --- | --- |
| 多租户隔离 | 按园区隔离；企业用户仅见本企业数据（数据级权限 + 行级过滤） |
| 入侵防护 | **WAF** + Sentinel 限流熔断 + 黑名单联动（closedmgmt 触发） |
| 可用性 | K8s 多副本 + 反亲和；日全量 + 实时增量备份；RTO ≤ 30min |
| 视频安全 | GB/T 28181 联网，视频流经网关鉴权，抓拍图落 MinIO 加密 |

**应急场景安全**：`emergency-svc` 独立资源配额，应急模式下优先级最高，调度链路不依赖外部服务可用性。

---

## 十、分期交付映射

| 阶段 | 优先上线服务 | 说明 |
| --- | --- | --- |
| **M1–M12（基础夯实）** | iam / mdm / iot-ingest / device / safety / monitor / closedmgmt / ioc / gateway / 数据中台基础 | 合规达标，安全六大子系统上线 |
| **M13–M24（能力提升）** | env / energy / emergency(补全 B11–B13) / ioc 升级(A3/A4) | 环保 + 能碳 + 数字孪生 + 应急联动 |
| **M25–M36（智慧赋能）** | operation(全量 F) / corridor(E4) / AI 大模型服务 / 无人化巡检 | 产业生态 + 智能赋能 |

---

## 附录：技术栈清单

| 层 | 技术选型 |
| --- | --- |
| 开发框架 | Spring Boot 3.x / Spring Cloud 2023.x |
| 服务治理 | Nacos（注册+配置）/ Spring Cloud Gateway / OpenFeign / Sentinel |
| 安全 | Spring Security + OAuth2.1/OIDC / JJWT |
| 数据 | PostgreSQL 14 + MyBatis-Plus / JPA；TimescaleDB（时序）；Redis 7 |
| 消息 | Kafka 3.x（主）+ RabbitMQ（内部通知兜底） |
| 实时计算 | Flink / Kafka Streams |
| 分布式事务 | Seata（Saga / TCC） |
| 工作流 | Flowable / Activiti（作业票、审批、应急流程） |
| 状态机 | Spring Statemachine（隐患/作业票流转） |
| 可观测 | Prometheus + Grafana + Micrometer + ELK |
| 部署 | Docker + Kubernetes + Helm；CI/CD（GitLab/Jenkins） |

| 层 | 技术选型 |
| --- | --- |
| 视频/AI | 视频网关对接 GB/T 28181；AI 推理独立 Python 服务（gRPC） |
| 对象存储 | MinIO |

# 二、服务接口契约与事件设计

> **版本**：V1.0 | **日期**：2026-08-22 **技术栈**：Java / Spring Boot 3.x + Spring Cloud + OpenAPI 3.0 + Kafka **配套文档**：《化工园区信息化平台-微服务架构设计方案.md》 **约定**：所有 REST 接口统一前缀 `/api/{service}`；请求/响应体均为 `application/json;charset=UTF-8`；时间字段统一 ISO-8601（UTC，如 `2026-08-22T10:30:00Z`）。

## 目录


- 第一部分 通用规范
- 第二部分 OpenAPI 接口契约
- 2.1 iam-svc 统一身份
- 2.2 mdm-svc 主数据
- 2.3 iot-ingest-svc 采集接入
- 2.4 device-svc 设备管理
- 2.5 safety-svc 安环管理
- 2.6 monitor-svc 重大危险源监测
- 2.7 closedmgmt-svc 封闭化/人员定位
- 2.8 env-svc 环保监管
- 2.9 energy-svc 能碳管理
- 2.10 emergency-svc 应急指挥
- 2.11 operation-svc 园区运营
- 2.12 ioc-svc 智能运营中心
- 第三部分 Kafka Topic 与事件 Schema
- 3.1 Topic 总览
- 3.2 核心事件 Schema
- 3.3 分区与保序策略
- 第四部分 版本与兼容策略


## 第一部分 通用规范

## 1.1 统一响应包络

```
{
  "code": 0,
  "message": "success",
  "traceId": "a1b2c3d4-...",
  "data": { },
  "timestamp": "2026-08-22T10:30:00Z"
}
```

| code | 含义 |
| --- | --- |
| 0 | 成功 |
| 400xx | 业务校验失败（参数/状态非法） |
| 401xx | 未认证/无权限 |
| 404xx | 资源不存在 |
| 409xx | 状态冲突（如重复提交） |
| 500xx | 服务内部错误 |

## 1.2 公共请求头

| Header | 必填 | 说明 |
| --- | --- | --- |
| `Authorization` | 是 | `Bearer {accessToken}` |
| `X-Tenant-Id` | 是 | 园区/租户 ID（多租户） |
| `X-Request-Id` | 否 | 幂等/链路追踪 |
| `Accept-Language` | 否 | `zh-CN` / `en-US` |

## 1.3 公共分页参数

`pageNum`（默认1）/ `pageSize`（默认20）/ `sort`（如 `createdAt,desc`）；响应含 `total`、`pages`、`list`。

## 1.4 公共错误码语义

- 隐患状态机非法跃迁 → `40901`
- 作业票审批冲突 → `40902`
- 设备离线无法下发指令 → `40903`
- 应急资源已被锁定 → `40904`

---

# 第二部分 OpenAPI 接口契约

> 以下以 **OpenAPI 3.0 片段 + 关键字段说明** 形式给出，可直接经 SpringDoc 生成完整 `openapi.yaml`。

## 2.1 iam-svc 统一身份

**Base**：`/api/iam`

| Method | Path | 说明 | 权限 |
|---|---|---|---|
| POST | `/auth/login` | 登录获取 token | 公开 |
| POST | `/auth/refresh` | 刷新 token | 登录态 |
| POST | `/auth/logout` | 强制下线 | 登录态 |
| GET | `/users` | 用户列表（模块/功能/数据级过滤） | admin |
| POST | `/users` | 创建用户 | admin |
| PUT | `/users/{id}` | 启用/禁用/重置密码 | admin |
| GET | `/roles` | 角色列表 | admin |
| POST | `/roles` | 创建角色（三级权限） | admin |
| GET | `/audit-logs` | 审计日志（只追加查询） | auditor |
| GET | `/me/permissions` | 当前用户权限范围 | 登录态 |

**User 实体**

```
{
  "id": "u_1001",
  "username": "zhangsan",
  "roleIds": ["r_safety"],
  "enterpriseId": "e_001",
  "dataScope": "ENTERPRISE",
  "status": "ENABLED",
  "lastLoginAt": "2026-08-22T09:00:00Z"
}
```

**Role 实体**（三级权限）

```
{
  "id": "r_safety",
```

```
  "name": "安全科",
  "modulePerms": ["safety:read", "monitor:read", "emergency:dispatch"],
  "funcPerms": ["hazard:approve", "ticket:review"],
  "dataLevel": "PARK_OR_ENTERPRISE"
}
```

## 2.2 mdm-svc 主数据

**Base**：`/api/mdm`

| Method | Path | 说明 |
| --- | --- | --- |
| GET | `/parks` | 园区列表 |
| GET | `/enterprises` | 企业一企一档（基本信息/危化品登记/证照） |
| GET | `/enterprises/{id}/licenses` | 企业证照（含有效期，支持临期查询） |
| GET | `/hazards` | 重大危险源列表（N:N 企业） |
| GET | `/hazards/{id}/sensors` | 危险源关联传感器 |
| POST | `/hazards` | 登记重大危险源 |
| GET | `/sensors/{id}` | 传感器主数据（位号/量程/采集频率） |
| GET | `/monitor-points` | 环保监测点主数据（HJ 协议地址） |

**Enterprise 实体**

```
{
  "id": "e_001",
  "name": "某化工有限公司",
  "creditCode": "91320000XXXX",
  "industry": "有机化工",
  "riskLevel": "RED",
  "license": { "type": "安全生产许可证", "expireAt": "2027-01-01" }
}
```

> 主数据为单一真相源；其他服务订阅快照到本地读模型，变更经 Saga 同步。

## 2.3 iot-ingest-svc 采集接入

**Base**：`/api/iot`

| Method | Path | 说明 |
| --- | --- | --- |
| POST | `/points/ingest` | 边缘网关上报点位（内部/网关调用） |
| GET | `/adapters` | 协议适配器状态（OPC UA/Modbus/MQTT/HJ212） |
| POST | `/adapters/{id}/config` | 接入配置（点位映射） |
| GET | `/devices/{id}/latest` | 设备最新原始值（透传） |

**IngestPayload**

```
{
  "deviceNo": "TANK-01-LT",
  "metric": "level",
  "value": 82.5,
  "unit": "%",
  "quality": "GOOD",
  "ts": "2026-08-22T10:30:00Z"
}
```

> 该服务将清洗后数据写入 `topic: iot.raw`，下游 monitor/env/energy/device 消费。

## 2.4 device-svc 设备管理

**Base**：`/api/device`

| Method | Path | 说明 |
| --- | --- | --- |
| GET | `/devices` | 设备台账（类型/状态/所属） |
| GET | `/devices/{id}` | 设备详情 + 健康度 |
| POST | `/devices` | 登记设备 |
| PUT | `/devices/{id}/maintenance` | 维保记录 |
| GET | `/devices/{id}/health` | 健康度（在线率/故障数） |
| GET | `/u-position/{cabinetId}` | 机柜 U 位识别（RFID + 台账融合） |

**Device 实体**

```
{
  "id": "d_2001",
  "no": "TANK-01-LT",
  "type": "LEVEL_TRANSMITTER",
  "cabinet": "C-03",
```

```
  "status": "ONLINE",
  "health": 96,
  "lastMaintenanceAt": "2026-07-01T00:00:00Z"
}
```

## 2.5 safety-svc 安环管理

**Base**：`/api/safety`

| Method | Path | 说明 | 状态机 |
|---|---|---|---|
| GET | `/risks` | 风险分级管控清单（红/橙/黄/蓝） | — |
| POST | `/risks` | 企业上传风险点 | — |
| GET | `/hazards` | 隐患台账 | — |
| POST | `/hazards` | 企业自查上报 | 初始 `PENDING` |
| POST | `/hazards/{id}/rectify` | 整改中 | `PENDING→RECTIFYING` |
| POST | `/hazards/{id}/submit` | 提交验收 | `RECTIFYING→VERIFYING` |
| POST | `/hazards/{id}/close` | 销号 | `VERIFYING→CLOSED` |
| GET | `/tickets` | 特殊作业票列表 | — |
| POST | `/tickets` | 八类作业申报 | `SUBMITTED` |
| POST | `/tickets/{id}/review` | 园区审核（驳回/通过） | `SUBMITTED→REVIEWED/REJECTED` |
| POST | `/tickets/{id}/approve` | 批准作业 | `REVIEWED→APPROVED` |
| GET | `/tickets/{id}/joint-check` | 联合抽查（调device+monitor+视频AI） | — |
| GET | `/enterprises/{id}/profile` | 一企一档 | — |

**Hazard 实体**（隐患排查治理）

```
{
  "id": "h_5001",
  "enterpriseId": "e_001",
  "level": "MAJOR",
  "status": "RECTIFYING",
  "dueAt": "2026-08-25T00:00:00Z",
  "description": "反应釜区静电跨接缺失"
}
```

> 隐患状态机：`PENDING→RECTIFYING→VERIFYING→CLOSED`；非法跃迁返回 `40901`。临期(48h/24h)/超期由定时任务扫描触发提醒事件。

## 2.6 monitor-svc 重大危险源监测

**Base**：`/api/monitor`

| Method | Path | 说明 |
| --- | --- | --- |
| GET | `/hazards/{id}/realtime` | 实时参数（液位/温度/压力/气体） |
| GET | `/hazards/{id}/history` | 历史趋势（拖拽时间轴） |
| GET | `/warnings` | 四级风险预警列表 |
| GET | `/warnings/{id}` | 预警详情（触发参数/阈值/趋势） |
| POST | `/warnings/{id}/verify` | 核查督办（指派/结果） |
| GET | `/rules` | 阈值规则（自定义） |
| POST | `/rules` | 新增/修改规则 |

**Warning 实体**

```
{
  "id": "w_7001",
  "hazardId": "hz_01",
  "level": "ORANGE",
  "metric": "pressure",
  "current": 1.85,
  "threshold": 1.6,
  "status": "VERIFYING",
  "triggeredAt": "2026-08-22T10:00:00Z"
}
```

> 超阈值 → 发布 `AlarmEvent` 到 Kafka；实时链路 10s 刷新（读写分离库）。

## 2.7 closedmgmt-svc 封闭化/人员定位

**Base**：`/api/closed`

| Method | Path | 说明 |
| --- | --- | --- |
| GET | `/gates` | 门禁/卡口状态（通行/黑名单拦截数） |

| Method | Path | 说明 |
|---|---|---|
| GET | `/access-records` | 出入记录检索导出 |
| POST | `/visitors` | 临时访客预约（二维码通行） |
| GET | `/persons/location` | 人员实时位置（UWB/蓝牙） |
| GET | `/persons/{id}/track` | 单人轨迹回放 |
| GET | `/persons/zone-count` | 区域人数统计 |
| POST | `/persons/sos` | SOS 一键求助 |
| GET | `/vehicles/track` | 危化品车辆 GPS 追踪 |
| GET | `/vehicles/{id}/fence` | 电子围栏偏离状态 |
| GET | `/parking` | 停车场占用 |

**PersonLocation 实体**

```
{
  "personId": "p_001",
  "enterpriseId": "e_001",
  "lng": 119.95, "lat": 32.01,
  "zone": "ZONE-A",
  "fenceBreach": false,
  "ts": "2026-08-22T10:30:00Z"
}
```

> 越界/ SOS → 发布 `EmergencyTriggeredEvent`；黑名单拦截 → `AlarmEvent`。

## 2.8 env-svc 环保监管

**Base**：`/api/env`

| Method | Path | 说明 |
|---|---|---|
| GET | `/air-quality` | 大气参数（$SO_2$/NOx/PM） |
| GET | `/water-quality` | 水质（COD/氨氮/pH） |
| GET | `/discharge/{id}/realtime` | 排口 HJ212 实时 |
| GET | `/discharge/{id}/history` | 排口历史 |
| GET | `/voc` | VOCs/厂界异味 |
| GET | `/solid-waste/manifest` | 固废转移联单 |

| Method | Path | 说明 |
| --- | --- | --- |
| POST | `/solid-waste/manifest` | 联单上报 |
| GET | `/source-trace` | 污染溯源（关联上下游+生产时段） |

> 超标 → `AlarmEvent`（环保类）；超 90 天未处置固废 → `ReminderEvent`。

## 2.9 energy-svc 能碳管理

**Base**：`/api/energy`

| Method | Path | 说明 |
| --- | --- | --- |
| GET | `/meters/realtime` | 多能源实时计量 |
| GET | `/carbon/scope` | Scope1/2/3 碳足迹 |
| GET | `/carbon/quota` | 碳配额（分配/使用/结余） |
| GET | `/efficiency/diagnosis` | 能效诊断 |
| GET | `/microgrid` | 源网荷储调度状态 |

**CarbonQuota 实体**

```
{
  "enterpriseId": "e_001",
  "allocated": 12000.0,
  "used": 8650.5,
  "balance": 3349.5,
  "unit": "tCO2e",
  "period": "2026"
}
```

## 2.10 emergency-svc 应急指挥

**Base**：`/api/emergency`

| Method | Path | 说明 |
| --- | --- | --- |
| GET | `/plans` | 数字化预案库（结构化检索） |
| POST | `/plans` | 预案编制/版本 |
| GET | `/resources` | 应急资源一张图（专家/物资/队伍/医院） |
| GET | `/resources/recommend` | 周边可用资源推荐（≤3km） |

| Method | Path | 说明 |
| --- | --- | --- |
| POST | `/incidents` | 接处警登记 |
| POST | `/incidents/{id}/dispatch` | 一键调度（语音/视频/短信/广播） |
| POST | `/incidents/{id}/conference` | 多方会商 |
| GET | `/incidents/{id}/timeline` | 处置时间轴 |
| POST | `/simulations` | 事故模拟推演（ALOHA/PHAST/FDS） |
| GET | `/simulations/{id}` | 推演结果（影响范围/疏散路线） |

**Incident 实体**

```
{
  "id": "inc_001",
  "type": "TOXIC_LEAK",
  "level": "LEVEL_I",
  "location": { "lng": 119.96, "lat": 32.02 },
  "status": "RESPONDING",
  "planId": "plan_023",
  "createdAt": "2026-08-22T10:05:00Z"
}
```

> 该服务对其他域调用均带熔断降级 + 本地预案基线兜底，保障"黄金十分钟"调度。

## 2.11 operation-svc 园区运营

**Base**：`/api/operation`

| Method | Path | 说明 |
| --- | --- | --- |
| POST | `/enterprises/apply` | 入园审批（申报→预审→联审→批复） |
| GET | `/policies/push` | 政策精准推送 |
| GET | `/logistics/vehicles` | 危化品车辆排队叫号 |
| GET | `/warehouse/{id}` | 危化品仓库温湿度/库存 |
| GET | `/industry-chain` | 上下游产能匹配 |
| POST | `/oa/documents` | 公文流转 |
| GET | `/economy/indicators` | 经济运行监测 |

## 2.12 ioc-svc 智能运营中心

**Base**：`/api/ioc`

| Method | Path | 说明 |
|---|---|---|
| GET | `/overview` | 综合态势一张图聚合（安全/环保/应急/能源/物流） |
| GET | `/risk-map` | 安全风险四色图 |
| GET | `/alarms` | 实时告警中心聚合 |
| GET | `/alarms/stats` | 告警多维统计 |
| POST | `/mode/switch` | 平急模式切换（≤3s） |
| GET | `/dashboard/emergency` | 应急指挥大屏数据 |

> `ioc-svc` 为只读聚合（CQRS 读侧）：订阅 Kafka 告警事件 + 主动 Feign 拉取各域摘要，不写业务库。

---

# 第三部分 Kafka Topic 与事件 Schema

## 3.1 Topic 总览

| Topic | 生产者 | 消费者 | 用途 | 分区 | 保留 |
|---|---|---|---|---|---|
| `iot.raw` | iot-ingest | monitor / env / energy / device | 原始点位流 | 12 | 7d |
| `alarm.event` | monitor / env / closedmgmt / corridor | alert-engine / notification / ioc / emergency | 多源告警聚合 | 6 | 30d |
| `warning.status` | monitor | ioc / safety（督办） | 预警状态变更 | 3 | 30d |
| `emergency.triggered` | closedmgmt / monitor / safety | emergency / notification / ioc | 应急触发 | 3 | 30d |
| `emergency.dispatch` | emergency | notification / closedmgmt / communication | 调度指令 | 3 | 14d |
| `hazard.status` | safety | ioc / mdm（统计） | 隐患状态流转 | 3 | 30d |
| `ticket.status` | safety | ioc / device（气体联动） | 作业票状态 | 3 | 30d |
| `resource.lock` | emergency | emergency（自身）/ audit | 资源锁定/释放 | 3 | 14d |
| `mdm.snapshot` | mdm | 各业务域读模型 | 主数据快照同步 | 3 | 7d |
| `reminder.event` | safety / env / iam | notification | 临期/超期提醒 | 3 | 14d |

| Topic | 生产者 | 消费者 | 用途 | 分区 | 保留 |
| --- | --- | --- | --- | --- | --- |
| `audit.log` | 所有服务 | ELK / audit-svc | 操作审计（WORM） | 6 | 365d |
| `simulation.result` | emergency(推演) | ioc | 推演结果 | 3 | 30d |

## 3.2 核心事件 Schema

所有事件统一包络：

```
{
  "eventId": "evt_20260822_a1b2",
  "eventType": "ALARM_RAISED",
  "source": "monitor-svc",
  "tenantId": "park_001",
  "occurredAt": "2026-08-22T10:30:00Z",
  "traceId": "trace_xxx",
  "payload": { }
}
```

### 3.2.1 AlarmEvent（`alarm.event`）

```
{
  "eventId": "evt_001",
  "eventType": "ALARM_RAISED",
  "source": "monitor-svc",
  "tenantId": "park_001",
  "occurredAt": "2026-08-22T10:30:00Z",
  "payload": {
    "alarmId": "al_9001",
    "category": "SAFETY",
    "level": 1,
    "subType": "PRESSURE_OVER",
    "refId": "hz_01",
    "refType": "HAZARD",
    "metric": "pressure",
    "value": 1.85,
    "threshold": 1.6,
    "location": { "lng": 119.95, "lat": 32.01 },
    "videoRef": "cam_07"
```

```
  }
}
```

`level`：1 红(立即处置)/2 橙(30min)/3 黄(2h)/4 蓝(记录)。分级由告警引擎依据规则执行；超时未处理触发升级事件。

### 3.2.2 EmergencyTriggeredEvent（`emergency.triggered`）

```
{
  "eventId": "evt_002",
  "eventType": "EMERGENCY_TRIGGERED",
  "source": "closedmgmt-svc",
  "tenantId": "park_001",
  "occurredAt": "2026-08-22T10:31:00Z",
  "payload": {
    "triggerType": "SOS",
    "personId": "p_001",
    "location": { "lng": 119.96, "lat": 32.02 },
    "autoRecommendPlan": true
  }
}
```

### 3.2.3 HazardStatusEvent（`hazard.status`）

```
{
  "eventId": "evt_003",
  "eventType": "HAZARD_STATUS_CHANGED",
  "source": "safety-svc",
  "tenantId": "park_001",
  "occurredAt": "2026-08-22T10:32:00Z",
  "payload": {
    "hazardId": "h_5001",
    "enterpriseId": "e_001",
    "from": "PENDING",
    "to": "RECTIFYING",
    "operator": "u_1001"
  }
}
```

### 3.2.4 IotRawEvent（`iot.raw`）

```
{
  "eventId": "evt_004",
  "eventType": "IOT_RAW",
  "source": "iot-ingest-svc",
  "tenantId": "park_001",
  "occurredAt": "2026-08-22T10:30:00Z",
  "payload": {
    "deviceNo": "TANK-01-LT",
    "metric": "level",
    "value": 82.5,
    "unit": "%",
    "quality": "GOOD",
    "ts": "2026-08-22T10:30:00Z"
  }
}
```

#### 3.2.5 MdmSnapshotEvent（`mdm.snapshot`）

```
{
  "eventId": "evt_005",
  "eventType": "MDM_SNAPSHOT",
  "source": "mdm-svc",
  "tenantId": "park_001",
  "occurredAt": "2026-08-22T10:00:00Z",
  "payload": {
    "entity": "HAZARD",
    "action": "UPSERT",
    "data": { "id": "hz_01", "name": "液氯储罐", "riskLevel": "RED" }
  }
}
```

### 3.3 分区与保序策略

- **保序维度**：以 `deviceNo` / `refId` / `hazardId` 作为分区键（Partition Key），保证同一设备/同一隐患的事件**严格有序**。
- **幂等消费**：所有消费者按 `eventId` 去重（Redis/本地表），重复投递安全。
- **至少一次投递**：Producer 开启 `acks=all` + 重试；消费者先落库再提交 offset。
- **死信队列**：消费失败超阈值进入 `*.DLT` 主题，告警人工介入。
- **Schema 演进**：事件体采用向后兼容扩展（新增字段带默认值），禁止修改/删除既有字段语义。

---

## 第四部分 版本与兼容策略

- **API 版本**：路径携带主版本（如 `/api/safety/v1`），破坏性变更升主版本，新旧并行一个发布周期。
- **事件 Schema**：遵循向后兼容（additive-only），消费方忽略未知字段；重大变更发新 Topic（如 `alarm.event.v2`）。
- **契约测试**：各服务 REST 契约经 Spring Cloud Contract / OpenAPI 校验，CI 阶段拦截不兼容变更。
- **网关兼容**：Gateway 按 `Accept-Version` 头或路径路由到对应版本实例，支持灰度。

---

# 三、分布式事务（Seata Saga）实现示例

---

> **版本**：V1.0 | **日期**：2026-08-22 **配套文档**：《化工园区信息化平台-微服务架构设计方案.md》《化工园区信息化平台-服务接口契约与事件设计.md》 **场景**：特殊作业票审批长流程（提交→气体检测→视频抽查→园区审核→批准作业），跨 `safety-svc` / `device-svc` / `monitor-svc` / 视频AI网关。

---

## 目录


1. 为什么用 Saga
2. 作业票审批流程定义
3. Saga 状态机 JSON 定义
4. Java 实现示例
5. 补偿（回滚）逻辑
6. 与业务状态机的配合
7. 异常处理与可观测


---

## 一、为什么用 Saga

特殊作业票审批是**跨多服务、长周期、可补偿**的业务：

- 提交作业票（`safety-svc` 本地事务）
- 联动查询气体检测实时值（`device-svc` + `monitor-svc`）
- 联动视频 AI 抽查（视频网关）
- 园区审核（人工，可能耗时数小时）
- 批准作业（状态变更）

若用 2PC（XA）会长时间锁资源、拖垮高频监测库。**Saga** 以"本地事务 + 前向/补偿"方式，保证最终一致且不长期持锁，符合化工园区"审批可驳回/可退回修改"的业务现实。

---

## 二、作业票审批流程定义

```
[Submit] ──→ [GasCheck] ──→ [VideoCheck] ──→ [Review] ──→ [Approve]
   |             |               |              |            |
   └补偿:        └补偿:          └补偿:         └补偿:       └补偿:
   rollbackDraft  clearGasHold    clearVideoHold  rejectToDraft revertApprove
```

| 步骤 | 服务 | 本地事务 | 补偿动作 |
| --- | --- | --- | --- |
| Submit | safety-svc | 插入作业票 `SUBMITTED` | 置 `DRAFT` / 逻辑删除 |
| GasCheck | device/monitor | 记录气体检测快照、占用传感器查询会话 | 释放查询会话、标记快照失效 |
| VideoCheck | 视频网关 | 创建抽查任务、截图 | 取消任务、删除临时截图 |
| Review | safety-svc | 状态 `REVIEWED` / `REJECTED` | 回退 `SUBMITTED` |
| Approve | safety-svc | 状态 `APPROVED`，作业生效 | 回退 `REVIEWED`，作业失效 |

> 任一前向步骤失败，Saga 逆序执行已完成步骤的补偿，使整体回到一致态（作业票回到 `DRAFT` 或保留在失败前状态并标记 `ABNORMAL`）。

---

## 三、Saga 状态机 JSON 定义

Seata Saga 使用状态机 JSON（`seata-saga-statemachine-designer` 可可视化编辑）。

```
{
  "Name": "hotWorkTicketApproval",
  "Comment": "特殊作业票(动火)审批 Saga",
  "StartState": "Submit",
  "Version": "1.0.0",
  "States": {
    "Submit": {
      "Type": "ServiceTask",
      "ServiceName": "safetyTicketService",
      "ServiceMethod": "submit",
      "CompensateState": "SubmitCompensate",
      "Next": "GasCheck",
      "Input": ["$.ticketId", "$.enterpriseId"]
    },
    "GasCheck": {
      "Type": "ServiceTask",
      "ServiceName": "gasCheckService",
      "ServiceMethod": "check",
      "CompensateState": "GasCheckCompensate",
```

```
      "Next": "VideoCheck",
      "Input": ["$.ticketId", "$.sensors"]
    },
    "VideoCheck": {
      "Type": "ServiceTask",
      "ServiceName": "videoCheckService",
      "ServiceMethod": "spotCheck",
      "CompensateState": "VideoCheckCompensate",
      "Next": "Review",
      "Input": ["$.ticketId", "$.cameraIds"]
    },
    "Review": {
      "Type": "ServiceTask",
      "ServiceName": "safetyTicketService",
      "ServiceMethod": "review",
      "CompensateState": "ReviewCompensate",
      "Next": "Approve",
      "Input": ["$.ticketId", "$.reviewerId"]
    },
    "Approve": {
      "Type": "ServiceTask",
      "ServiceName": "safetyTicketService",
      "ServiceMethod": "approve",
      "CompensateState": "ApproveCompensate",
      "Next": "Success",
      "Input": ["$.ticketId", "$.approverId"]
    },
    "SubmitCompensate": { "Type": "ServiceTask", "ServiceName": "safetyTicketService", "ServiceMet
    "GasCheckCompensate": { "Type": "ServiceTask", "ServiceName": "gasCheckService", "ServiceMetho
    "VideoCheckCompensate": { "Type": "ServiceTask", "ServiceName": "videoCheckService", "ServiceM
    "ReviewCompensate": { "Type": "ServiceTask", "ServiceName": "safetyTicketService", "ServiceMet
    "ApproveCompensate": { "Type": "ServiceTask", "ServiceName": "safetyTicketService", "ServiceMe
    "Success": { "Type": "Succeed" },
    "Fail": { "Type": "Fail" }
  }
}
```

> 说明：`Input` 使用 JsonPath 从上下文取参；`CompensateState` 声明失败时逆序回滚入口。

---

## 四、Java 实现示例

## 4.1 配置（Seata Saga）

```
# application.yml (safety-svc)
seata:
  enabled: true
  tx-service-group: safety_tx_group
  saga:
    state-machine-mode: true
  config:
    type: nacos
  registry:
    type: nacos
```

## 4.2 发起 Saga（safety-svc）

```
@Service
@RequiredArgsConstructor
public class TicketApprovalAppService {

    private final StateMachineEngine stateMachineEngine;
    private final TicketRepository ticketRepository;

    public String startApproval(HotWorkTicketCmd cmd) {
        // 1. 先落本地草稿（独立本地事务，保证 ticketId 存在）
        Ticket ticket = ticketRepository.save(cmd.toDraft());

        // 2. 构造 Saga 上下文
        Map<String, Object> ctx = new HashMap<>();
        ctx.put("ticketId", ticket.getId());
        ctx.put("enterpriseId", cmd.getEnterpriseId());
        ctx.put("sensors", cmd.getGasSensorNos());   // 如 ["TANK-01-GT","TANK-02-GT"]
        ctx.put("cameraIds", cmd.getCameraIds());    // 如 ["cam_07"]
        ctx.put("reviewerId", cmd.getReviewerId());
        ctx.put("approverId", cmd.getApproverId());

        // 3. 启动 Saga（异步，状态变更由各服务本地事务提交）
        String businessKey = "TICKET_" + ticket.getId();
        stateMachineEngine.startWithBusinessKey("hotWorkTicketApproval",
                null, businessKey, ctx);
        return businessKey;
    }
}
```

### 4.3 各服务 Task 实现（示例：GasCheck）

```
// gasCheckService 由 device-svc / monitor-svc 通过 Feign 暴露，Saga 引擎经注册中心发现
@Component("gasCheckService")
public class GasCheckService {

    @Autowired
    private DeviceClient deviceClient;     // Feign → device-svc
    @Autowired
    private MonitorClient monitorClient;   // Feign → monitor-svc
    @Autowired
    private GasSnapshotRepository repo;

    public boolean check(Map<String, Object> ctx) {
        String ticketId = (String) ctx.get("ticketId");
        List<String> sensors = (List<String>) ctx.get("sensors");

        // 取实时气体浓度（可燃/有毒），并做合格判定
        for (String s : sensors) {
            RealtimeVO vo = monitorClient.getRealtime(s);
            if (vo.getValue() > vo.getThreshold()) {
                throw new BizException("GAS_OVER_LIMIT", s + " 浓度超标，禁止作业");
            }
            repo.save(new GasSnapshot(ticketId, s, vo.getValue(), vo.getTs()));
        }
        return true;
    }

    public boolean compensateCheck(Map<String, Object> ctx) {
        // 释放查询会话、标记快照失效（幂等）
        repo.invalidateByTicket((String) ctx.get("ticketId"));
        return true;
    }
}
```

> `videoCheckService` / `safetyTicketService` 结构类似，分别对接视频网关与本地作业票库。

---

## 五、补偿（回滚）逻辑

补偿遵循三条原则：

1. **幂等**：补偿可因重试多次执行，须以业务键（ticketId）去重，不产生副作用。
2. **可逆**：前向动作必须有对应补偿；如"批准作业"补偿为"回退到 REVIEWED 且作业失效"。
3. **不破坏审计**：补偿仅改作业票**状态**与**占用资源释放**，所有操作仍入审计日志（WORM），保留完整链路。

```
// safetyTicketService 补偿示例
@Component("safetyTicketService")
public class SafetyTicketService {

    public boolean compensateSubmit(Map<String, Object> ctx) {
        String id = (String) ctx.get("ticketId");
        Ticket t = repo.findById(id).orElse(null);
        if (t == null) return true;                  // 已删除，幂等返回
        if (t.getStatus() == TicketStatus.SUBMITTED) {
            t.setStatus(TicketStatus.DRAFT);         // 回到草稿
            repo.save(t);
        }
        return true;
    }

    public boolean compensateApprove(Map<String, Object> ctx) {
        String id = (String) ctx.get("ticketId");
        Ticket t = repo.findById(id).orElse(null);
        if (t != null && t.getStatus() == TicketStatus.APPROVED) {
            t.setStatus(TicketStatus.REVIEWED);      // 作业失效，回退审核态
            t.setEffective(false);
            repo.save(t);
        }
        return true;
    }
}
```

---

## 六、与业务状态机的配合

Saga 负责**跨服务最终一致**，业务状态机（`Spring Statemachine`）负责**单服务内状态合法跃迁**，两者职责分离：

```
Saga 步骤成功 → 调用 safety-svc 内部状态机执行 TRANSITION
                (如 SUBMITTED → REVIEWED，由状态机守卫校验)
```

- Saga 不感知业务细节，只触发"服务方法"；

- 状态机拦截非法跃迁（如 `DRAFT → APPROVED` 跳过审核），抛出 `40901`，Saga 触发补偿；
- 人工审核/批准由用户操作触发对应 Saga 步骤（流程可暂停等待人工，Seata Saga 支持 `Wait` 状态 + 唤醒）。

```
// 审核步骤（人工唤醒）
public void review(String ticketId, String reviewerId, boolean pass) {
    // 状态机守卫：仅 SUBMITTED 可转 REVIEWED/REJECTED
    stateMachine.sendEvent(new ReviewEvent(ticketId, reviewerId, pass));
    // 通知 Saga 引擎继续/补偿
    sagaEngine.wakeUp(businessKeyOf(ticketId), Map.of("reviewPass", pass));
}
```

---

## 七、异常处理与可观测

| 场景 | 处理 |
| --- | --- |
| 前向步骤超时（如视频网关不可用） | Saga 标记 `ABNORMAL`，逆序补偿；作业票回到 `DRAFT` 并推送提醒 |
| 人工审核驳回 | 直接走 `ReviewCompensate + SubmitCompensate` 回到 `DRAFT`，企业可修改重报 |
| 补偿失败 | 进入 Saga 死信，告警运维；支持人工重试补偿 |
| 可观测 | Saga 实例状态、步骤耗时经 Prometheus 暴露；关键节点入 `audit.log` Topic |

**监控指标建议**：`saga_instances_total{state}`、`saga_step_duration`、`saga_compensate_total`，对接 Grafana 大屏。

---

# 四、时序数据库（TimescaleDB）表结构设计

> **版本**：V1.0 | **日期**：2026-08-22 **配套文档**：《化工园区信息化平台-微服务架构设计方案.md》《化工园区信息化平台-服务接口契约与事件设计.md》 **引擎**：PostgreSQL 14 + TimescaleDB 2.x（超表 + 连续聚合） **用途**：存储 IoT 高频点位、重大危险源实时参数、环保监测、能碳计量等时序数据；支撑 10s 刷新、历史回溯、趋势统计。

---

## 目录

1. 设计原则
2. 核心超表（Hypertable）
3. 连续聚合（Continuous Aggregate）
4. 保留策略与压缩
5. 关键索引与查询示例
6. 与业务库的关系
7. 多租户与分区

---

## 一、设计原则

- **窄表宽列**：以 `time` 为分区维，`device_no` / `metric` 为标签（tag），`value` + `quality` 为值。
- **标签字典化**：设备/指标元数据存业务库（mdm/device），时序库只存编码，避免宽表膨胀。
- **超表分块**：按时间（如 1 天）自动分块（chunk），热数据在高速存储、冷数据压缩/降采样。
- **连续聚合**：预计算 1min/5min/1h rollup，大屏与统计直接查聚合表，减负原始超表。
- **只追加**：时序数据为不可变事实，更新极少；点位修正走"补数"而非 UPDATE。

---

## 二、核心超表（Hypertable）

### 2.1 设备点位原始时序 `ts_device_metric`

```
CREATE TABLE ts_device_metric (
    time        TIMESTAMPTZ NOT NULL,
    tenant_id   VARCHAR(32)  NOT NULL,        -- 园区/租户
    device_no   VARCHAR(64)  NOT NULL,         -- 设备位号(关联 device-svc)
    metric      VARCHAR(32)  NOT NULL,         -- level/temp/pressure/gas...
    value       DOUBLE PRECISION NOT NULL,     -- 数值
    unit        VARCHAR(16),
    quality     SMALLINT NOT NULL DEFAULT 0,   -- 0 GOOD/1 BAD/2 UNCERTAIN (OPC UA)
    source      VARCHAR(16)  NOT NULL DEFAULT 'OPC_UA'
);

SELECT create_hypertable('ts_device_metric', 'time',
       chunk_time_interval => INTERVAL '1 day',
       partitioning_column => 'tenant_id',    -- 多租户子分区
       number_partitions => 4);
```

> 对应 Kafka `iot.raw` 落库；点位由 `iot-ingest-svc` 写入。

### 2.2 重大危险源实时参数 `ts_hazard_param`

（从设备点位按危险源维度投影，便于 B2 看板直查）

```
CREATE TABLE ts_hazard_param (
    time        TIMESTAMPTZ NOT NULL,
    tenant_id   VARCHAR(32) NOT NULL,
    hazard_id   VARCHAR(32) NOT NULL,          -- 重大危险源(关联 mdm)
    metric      VARCHAR(32) NOT NULL,          -- 液位/温度/压力/可燃/有毒
    value       DOUBLE PRECISION NOT NULL,
    threshold   DOUBLE PRECISION,              -- 阈值(冗余存便于越限判定)
    is_alarm    BOOLEAN DEFAULT FALSE
);

SELECT create_hypertable('ts_hazard_param', 'time',
       chunk_time_interval => INTERVAL '1 day',
       partitioning_column => 'tenant_id', number_partitions => 4);
```

## 2.3 环保监测 `ts_env_monitor`

```
CREATE TABLE ts_env_monitor (
    time        TIMESTAMPTZ NOT NULL,
    tenant_id   VARCHAR(32) NOT NULL,
    point_id    VARCHAR(32) NOT NULL,          -- 监测点(HJ212 地址，关联 env-svc)
    category    VARCHAR(16) NOT NULL,          -- AIR/WATER/NOISE/VOC
    metric      VARCHAR(32) NOT NULL,          -- COD/NH3/SO2/PM25...
    value       DOUBLE PRECISION NOT NULL,
    std_limit   DOUBLE PRECISION,              -- 排放标准限值
    is_over     BOOLEAN DEFAULT FALSE
);

SELECT create_hypertable('ts_env_monitor', 'time',
       chunk_time_interval => INTERVAL '1 day',
       partitioning_column => 'tenant_id', number_partitions => 4);
```

## 2.4 能源与碳排 `ts_energy_meter`

```
CREATE TABLE ts_energy_meter (
    time        TIMESTAMPTZ NOT NULL,
    tenant_id   VARCHAR(32) NOT NULL,
    enterprise_id VARCHAR(32) NOT NULL,        -- 企业(关联 mdm)
    energy_type VARCHAR(16) NOT NULL,          -- ELEC/WATER/STEAM/GAS
    value       DOUBLE PRECISION NOT NULL,     -- 计量值
    power       DOUBLE PRECISION,              -- 实时功率(电)
```

```
    carbon_equiv DOUBLE PRECISION            -- 折算碳当量( Scope 核算用)
);

SELECT create_hypertable('ts_energy_meter', 'time',
       chunk_time_interval => INTERVAL '1 day',
       partitioning_column => 'tenant_id', number_partitions => 4);
```

### 2.5 定位轨迹 `ts_location_track`

（人员 UWB / 车辆 GPS，写入频率高，单独超表）

```
CREATE TABLE ts_location_track (
    time        TIMESTAMPTZ NOT NULL,
    tenant_id   VARCHAR(32) NOT NULL,
    obj_type    VARCHAR(8)  NOT NULL,         -- PERSON/VEHICLE
    obj_id      VARCHAR(32) NOT NULL,
    lng         DOUBLE PRECISION NOT NULL,
    lat         DOUBLE PRECISION NOT NULL,
    zone        VARCHAR(32),
    fence_breach BOOLEAN DEFAULT FALSE
);

SELECT create_hypertable('ts_location_track', 'time',
       chunk_time_interval => INTERVAL '1 day',
       partitioning_column => 'tenant_id', number_partitions => 4);
```

---

## 三、连续聚合（Continuous Aggregate）

预计算大屏与统计所需 rollup，避免实时扫描原始超表。

### 3.1 设备点位 5 分钟聚合

```
CREATE MATERIALIZED VIEW v_device_metric_5m
WITH (timescaledb.continuous) AS
SELECT
    tenant_id,
    device_no,
    metric,
    time_bucket('5 minutes', time) AS bucket,
    avg(value)  AS avg_val,
    max(value)  AS max_val,
```

```
    min(value)  AS min_val,
    count(*)    AS cnt
FROM ts_device_metric
GROUP BY tenant_id, device_no, metric, bucket;

-- 每 1 分钟刷新
SELECT add_continuous_aggregate_policy('v_device_metric_5m',
       start_offset => INTERVAL '1 hour',
       end_offset   => INTERVAL '1 minute',
       schedule_interval => INTERVAL '1 minute');
```

### 3.2 重大危险源 1 分钟聚合（供 B2 看板）

```
CREATE MATERIALIZED VIEW v_hazard_param_1m
WITH (timescaledb.continuous) AS
SELECT
    tenant_id, hazard_id, metric,
    time_bucket('1 minute', time) AS bucket,
    avg(value) AS avg_val, max(value) AS max_val,
    bool_or(is_alarm) AS has_alarm
FROM ts_hazard_param
GROUP BY tenant_id, hazard_id, metric, bucket;

SELECT add_continuous_aggregate_policy('v_hazard_param_1m',
       start_offset => INTERVAL '2 hours',
       end_offset   => INTERVAL '1 minute',
       schedule_interval => INTERVAL '1 minute');
```

### 3.3 环保/能碳小时聚合

```
CREATE MATERIALIZED VIEW v_env_1h
WITH (timescaledb.continuous) AS
SELECT tenant_id, point_id, category, metric,
       time_bucket('1 hour', time) AS bucket,
       avg(value) AS avg_val, max(value) AS max_val,
       bool_or(is_over) AS has_over
FROM ts_env_monitor
GROUP BY tenant_id, point_id, category, metric, bucket;

CREATE MATERIALIZED VIEW v_energy_1h
WITH (timescaledb.continuous) AS
SELECT tenant_id, enterprise_id, energy_type,
```

```
       time_bucket('1 hour', time) AS bucket,
       sum(value) AS total, avg(power) AS avg_power,
       sum(carbon_equiv) AS carbon_total
FROM ts_energy_meter
GROUP BY tenant_id, enterprise_id, energy_type, bucket;
```

---

# 四、保留策略与压缩

```
-- 原始点位：热 30 天 → 压缩；冷数据降采样后保留
SELECT add_compression_policy('ts_device_metric', INTERVAL '7 days');
SELECT add_retention_policy ('ts_device_metric', INTERVAL '180 days');

SELECT add_compression_policy('ts_hazard_param', INTERVAL '7 days');
SELECT add_retention_policy ('ts_hazard_param', INTERVAL '365 days');

SELECT add_compression_policy('ts_env_monitor', INTERVAL '7 days');
SELECT add_retention_policy ('ts_env_monitor', INTERVAL '365 days');

SELECT add_compression_policy('ts_location_track', INTERVAL '3 days');
SELECT add_retention_policy ('ts_location_track', INTERVAL '90 days');

-- 连续聚合：5min 保留 30 天，1h 长期保留
SELECT add_retention_policy ('v_device_metric_5m', INTERVAL '30 days');
SELECT add_retention_policy ('v_energy_1h', INTERVAL '730 days');
```

> 压缩采用 TimescaleDB 原生列存（列式压缩 + 字典编码），点位类数据可压缩 10x+。

---

# 五、关键索引与查询示例

## 5.1 索引

TimescaleDB 对 `time` 自动建分区索引；业务查询补充复合索引：

```
CREATE INDEX idx_device_metric_dm
  ON ts_device_metric (tenant_id, device_no, metric, time DESC);

CREATE INDEX idx_hazard_param_hm
  ON ts_hazard_param (tenant_id, hazard_id, metric, time DESC);
```

```
CREATE INDEX idx_env_point
  ON ts_env_monitor (tenant_id, point_id, time DESC);

CREATE INDEX idx_location_obj
  ON ts_location_track (tenant_id, obj_type, obj_id, time DESC);
```

## 5.2 查询示例

### ① 重大危险源最近 1 小时曲线（B2 看板，10s 刷新）

```
SELECT time, metric, value, is_alarm
FROM ts_hazard_param
WHERE tenant_id = $1 AND hazard_id = $2
  AND time > now() - INTERVAL '1 hour'
ORDER BY time DESC;
```

### ② 历史趋势回溯（拖拽时间轴）

```
SELECT time_bucket('1 minute', time) AS t, avg(value) AS v
FROM ts_hazard_param
WHERE tenant_id = $1 AND hazard_id = $2 AND metric = $3
  AND time BETWEEN $4 AND $5
GROUP BY t ORDER BY t;
```

### ③ 环保超标统计（C2 溯源/统计）

```
SELECT point_id, count(*) FILTER (WHERE is_over) AS over_cnt,
       count(*) AS total_cnt
FROM ts_env_monitor
WHERE tenant_id = $1 AND time > now() - INTERVAL '1 day'
GROUP BY point_id;
```

### ④ 区域人数实时统计（B8）

```
SELECT zone, count(DISTINCT obj_id) AS person_cnt
FROM ts_location_track
WHERE tenant_id = $1 AND obj_type = 'PERSON'
  AND time > now() - INTERVAL '1 minute'
GROUP BY zone;
```

---

## 六、与业务库的关系

```
业务库(PostgreSQL)                    时序库(TimescaleDB)
──────────────────                    ──────────────────
mdm: 设备/危险源主数据      ──1:N──▶ ts_device_metric.device_no
device: 设备档案/健康        ──1:N──▶ ts_hazard_param / ts_location_track
monitor: 预警/阈值规则       ──关联──▶ ts_hazard_param (is_alarm 派生)
env: 监测点/排口             ──1:N──▶ ts_env_monitor.point_id
energy: 碳配额/核算          ──关联──▶ ts_energy_meter (carbon_equiv 汇总)
```

- **主数据在业务库，时序只存编码**：设备名/量程/责任人等从业务库 JOIN 获取，避免冗余。
- **告警派生**：`monitor-svc` 消费 `iot.raw`，比对阈值后写业务库 `warning` 表，同时可在时序写 `is_alarm` 便于检索。
- **回溯联动**：历史告警详情页同时查业务库（告警记录）+ 时序库（参数曲线）。

---

## 七、多租户与分区

- **单园区私有化**：`tenant_id` 可固定，超表仅按 `time` 分区即可。
- **SaaS 多园区**：`tenant_id` 作为子分区键（见建表语句 `partitioning_column`），配合行级权限（JDBC Filter 注入 `tenant_id`）实现数据隔离。
- **写入路径**：`iot-ingest-svc` → Kafka `iot.raw` → 消费者批量 `COPY`/INSERT 落超表；批量写入降低 WAL 压力。
- **读路径**：大屏 `ioc-svc` 直查连续聚合视图，监控服务查原始超表（带时间下推）。

---

# 五、Kubernetes Helm 部署清单示例

> **版本**：V1.0 | **日期**：2026-08-22 **配套文档**：《化工园区信息化平台-微服务架构设计方案.md》 **目标**：私有化/园区本地 K8s 集群一键部署 14 个微服务 + 中间件，满足等保三级与高可用。

---

## 目录


1. 集群与命名空间规划
2. Chart 目录结构
3. 全局 values.yaml
4. 中间件部署（PostgreSQL/TimescaleDB/Kafka/Redis/Nacos/MinIO）
5. 微服务 Deployment + Service 示例

6. Ingress / Gateway 暴露
7. HPA / 资源配额 / 反亲和
8. 安全上下文与网络策略
9. 部署命令

---

## 一、集群与命名空间规划

| 命名空间 | 用途 |
| --- | --- |
| `chem-park-middleware` | Nacos / Kafka / PostgreSQL / TimescaleDB / Redis / MinIO / ELK |
| `chem-park-app` | 14 个微服务 + Gateway |
| `chem-park-monitor` | Prometheus / Grafana / Alertmanager |
| `ingress-nginx` | Ingress 控制器 |

> 关键服务（monitor / emergency / ioc）打 `priorityClassName: high-priority`，OOM/驱逐时优先保留。

---

## 二、Chart 目录结构

```
chem-park-platform/
├── Chart.yaml
├── values.yaml
├── templates/
│   ├── namespace.yaml
│   ├── middleware/
│   │   ├── postgres.yaml
│   │   ├── timescaledb.yaml
│   │   ├── kafka.yaml
│   │   ├── redis.yaml
│   │   ├── nacos.yaml
│   │   └── minio.yaml
│   ├── gateway/
│   │   ├── deployment.yaml
│   │   └── service.yaml
│   ├── services/
│   │   ├── _service.tpl          # 通用微服务模板
│   │   ├── iam.yaml
│   │   ├── mdm.yaml
```

```
│   │   ├── iot-ingest.yaml
│   │   ├── device.yaml
│   │   ├── safety.yaml
│   │   ├── monitor.yaml
│   │   ├── closedmgmt.yaml
│   │   ├── env.yaml
│   │   ├── energy.yaml
│   │   ├── corridor.yaml
│   │   ├── emergency.yaml
│   │   ├── operation.yaml
│   │   └── ioc.yaml
│   ├── ingress.yaml
│   ├── hpa.yaml
│   ├── networkpolicy.yaml
│   └── pdb.yaml
└── charts/                       # 中间件子 Chart（bitnami）
```

## 三、全局 values.yaml

```yaml
global:
  imageRegistry: "registry.chem-park.local"
  imagePullSecrets: ["chem-park-registry"]
  tenantId: "park_001"
  storageClass: "local-ssd"        # 高速存储(时序/PG)
  backupStorageClass: "nfs-backup"

domain: "ioc.chem-park.example"

gateway:
  replicas: 2
  resources:
    requests: { cpu: "500m", memory: "512Mi" }
    limits:   { cpu: "1",    memory: "1Gi" }

services:
  # 通用默认；各服务可覆盖
  resources:
    requests: { cpu: "250m", memory: "512Mi" }
    limits:   { cpu: "1",    memory: "1Gi" }
  javaOpts: "-Xms512m -Xmx1g -XX:+UseG1GC -Dfile.encoding=UTF-8"
```

```
  # 高频/关键服务单独配置
  monitor:
    replicas: 3
    priorityClass: high-priority
    resources:
      requests: { cpu: "500m", memory: "1Gi" }
      limits:   { cpu: "2",    memory: "2Gi" }
  emergency:
    replicas: 2
    priorityClass: high-priority
  ioc:
    replicas: 3
    priorityClass: high-priority

middleware:
  postgres:   { replicas: 2, storage: "100Gi" }
  timescaledb:{ replicas: 2, storage: "500Gi" }
  kafka:      { brokers: 3 }
  redis:      { replicas: 3, mode: sentinel }
  nacos:      { replicas: 3 }
  minio:      { replicas: 4, storage: "1Ti" }
```

---

# 四、中间件部署

## 4.1 Nacos（注册 + 配置中心，3 节点）

```
# templates/middleware/nacos.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: nacos
  namespace: chem-park-middleware
spec:
  serviceName: nacos-headless
  replicas: 3
  selector:
    matchLabels: { app: nacos }
  template:
    metadata:
      labels: { app: nacos }
    spec:
```

```
    containers:
      - name: nacos
        image: "{{ .Values.global.imageRegistry }}/nacos-server:2.3.2"
        ports:
          - { containerPort: 8848, name: http }
          - { containerPort: 7848, name: raft }
        env:
          - { name: MODE, value: "cluster" }
          - { name: NACOS_REPLICAS, value: "3" }
          - { name: MYSQL_SERVICE_HOST, value: "postgres-middleware" }
        readinessProbe:
          httpGet: { path: /nacos/health/ready, port: 8848 }
          initialDelaySeconds: 30
        volumeMounts:
          - { name: data, mountPath: /home/nacos/data }
    volumes:
      - name: data
        persistentVolumeClaim:
          claimName: nacos-data
```

## 4.2 Kafka（3 broker + 副本）

```
# templates/middleware/kafka.yaml (摘要)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kafka
  namespace: chem-park-middleware
spec:
  replicas: 3
  serviceName: kafka-headless
  template:
    spec:
      containers:
        - name: kafka
          image: "{{ .Values.global.imageRegistry }}/bitnami/kafka:3.7"
          env:
            - { name: KAFKA_CFG_NUM_PARTITIONS, value: "12" }
            - { name: KAFKA_CFG_DEFAULT_REPLICATION_FACTOR, value: "3" }
            - { name: KAFKA_CFG_MIN_INSYNC_REPLICAS, value: "2" }
            - { name: KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE, value: "false" }
```

```
          ports:
            - { containerPort: 9092, name: tcp }
```

> `min.insync.replicas=2` + `acks=all` 保障事件不丢（见事件设计文档）。

### 4.3 TimescaleDB / PostgreSQL

```
# templates/middleware/timescaledb.yaml (摘要)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: timescaledb
  namespace: chem-park-middleware
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: timescaledb
          image: "{{ .Values.global.imageRegistry }}/timescale/timescaledb:2.14-pg14"
          env:
            - { name: POSTGRES_PASSWORD, valueFrom:
                secretKeyRef: { name: db-secret, key: ts-pass } }
          volumeMounts:
            - { name: pgdata, mountPath: /var/lib/postgresql/data }
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "postgres"]
```

---

## 五、微服务 Deployment + Service 示例

### 5.1 通用模板 `_service.tpl`

```
{{- define "chem.service" -}}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .name }}
  namespace: chem-park-app
```

```
  labels: { app: {{ .name }} }
spec:
  replicas: {{ .replicas | default 1 }}
  {{- if .priorityClass }}
  template:
    metadata:
      labels: { app: {{ .name }} }
    spec:
      priorityClassName: {{ .priorityClass }}
  {{- else }}
  template:
    metadata:
      labels: { app: {{ .name }} }
    spec:
  {{- end }}
      containers:
        - name: {{ .name }}
          image: "{{ $.Values.global.imageRegistry }}/{{ .name }}:{{ .tag | default "latest" }}"
          ports:
            - { containerPort: 8080 }
          env:
            - { name: NACOS_ADDR, value: "nacos.chem-park-middleware:8848" }
            - { name: KAFKA_ADDR, value: "kafka.chem-park-middleware:9092" }
            - { name: REDIS_ADDR, value: "redis.chem-park-middleware:6379" }
            - { name: TENANT_ID, value: "{{ $.Values.global.tenantId }}" }
            - { name: JAVA_OPTS, value: "{{ $.Values.services.javaOpts }}" }
          envFrom:
            - secretRef: { name: app-secret }
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
            initialDelaySeconds: 20
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8080 }
            initialDelaySeconds: 40
          resources:
            requests: {{ toYaml .resources.requests | nindent 14 }}
            limits:   {{ toYaml .resources.limits | nindent 14 }}
          securityContext:
            runAsNonRoot: true
            readOnlyRootFilesystem: true
            allowPrivilegeEscalation: false
---
apiVersion: v1
kind: Service
```

```
metadata:
  name: {{ .name }}
  namespace: chem-park-app
spec:
  selector: { app: {{ .name }} }
  ports:
    - { port: 8080, targetPort: 8080 }
{{- end -}}
```

### 5.2 monitor-svc 实例化（高频隔离）

```
# templates/services/monitor.yaml
{{- include "chem.service" (dict
    "name" "monitor-svc"
    "replicas" .Values.services.monitor.replicas
    "priorityClass" .Values.services.monitor.priorityClass
    "resources" .Values.services.monitor.resources
    "tag" "v1.3"
    "Values" .Values) -}}
```

> `emergency-svc` / `ioc-svc` 同理，均带 `high-priority` 与多副本。

---

## 六、Ingress / Gateway 暴露

```
# templates/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: chem-park-ingress
  namespace: chem-park-app
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "30"
    nginx.ingress.kubernetes.io/limit-rps: "50"
spec:
  ingressClassName: nginx
  tls:
    - hosts: ["{{ .Values.domain }}"]
      secretName: chem-park-tls
```

```
  rules:
    - host: "{{ .Values.domain }}"
      http:
        paths:
          - { path: /api/iam, pathType: Prefix, backend: { service: { name: gateway-svc, port: { n
          - { path: /api/safety, pathType: Prefix, backend: { service: { name: gateway-svc, port:
          - { path: /api/emergency, pathType: Prefix, backend: { service: { name: gateway-svc, por
          - { path: /api/ioc, pathType: Prefix, backend: { service: { name: gateway-svc, port: { n
```

> 外部统一走 `gateway-svc`（Spring Cloud Gateway），由网关做路由/鉴权/限流；Ingress 仅做 TLS 终止与域名暴露。

---

## 七、HPA / 资源配额 / 反亲和

```
# templates/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: monitor-hpa
  namespace: chem-park-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: monitor-svc
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
---
# 反亲和：同服务 Pod 打散到不同节点
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
```

```
          matchLabels: { app: monitor-svc }
        topologyKey: kubernetes.io/hostname
```

## 八、安全上下文与网络策略

```
# templates/networkpolicy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: app-isolate
  namespace: chem-park-app
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: chem-park-app } }
        - podSelector: { matchLabels: { app: gateway-svc } }
      ports:
        - { protocol: TCP, port: 8080 }
  egress:
    - to:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: chem-park-middleware }
    - to:
        - podSelector: { matchLabels: { app: kafka } }
```

> 默认拒绝跨命名空间流量，仅放行 Gateway 入向与中间件出向，满足等保网络隔离。

## 九、部署命令

```
# 1. 添加中间件子 Chart 依赖
helm dependency build chem-park-platform

# 2. 渲染检查
helm template chem-park-platform -n chem-park-app --create-namespace > render.yaml

# 3. 部署（私有化）
```

```
helm upgrade --install chem-park chem-park-platform \
  -n chem-park-app --create-namespace \
  -f values.yaml -f values-prod.yaml

# 4. 验证
kubectl get pods -n chem-park-app
kubectl get pods -n chem-park-middleware

# 5. 滚动升级（灰度）
helm upgrade chem-park chem-park-platform --set services.monitor.tag=v1.4 -n chem-park-app

# 6. 备份（PG/TimescaleDB 定时）
kubectl exec -n chem-park-middleware timescaledb-0 -- \
  pg_dumpall | gzip > /backup/ts-$(date +%F).sql.gz
```

---

# 六、等保三级安全加固检查清单

> **版本**：V1.0 | **日期**：2026-08-22 **依据**：GB/T 22239-2019《信息安全技术 网络安全等级保护基本要求》第三级 **配套文档**：《化工园区信息化平台-微服务架构设计方案.md》 **用途**：上线前安全自查与测评配合，覆盖"安全通信网络、安全区域边界、安全计算环境、安全管理中心"及 10 个控制类。

---

## 目录


1. 适用范围与定级
2. 安全物理环境（摘要）
3. 安全通信网络
4. 安全区域边界
5. 安全计算环境
6. 安全管理中心
7. 安全管理制度/机构/人员/建设/运维
8. 平台专项检查表（逐项打勾）
9. 测评配合建议


---

## 一、适用范围与定级

- **系统名称**：化工园区安全环保一体化智慧管控平台

- **定级**：第三级（等保 2.0 三级），属"第三级信息系统"
- **保护对象**：园区/企业安全环保数据、重大危险源实时数据、应急指挥链路
- **测评维度**：技术要求（通信网络/区域边界/计算环境/管理中心）+ 管理要求（5 类）

---

## 二、安全物理环境（摘要）

> 私有化部署于园区机房或政务云，物理环境由基础设施提供方保障，平台侧需确认：

- [ ] 机房访问控制（门禁+录像）、防盗、防火、防水、防雷、温湿度
- [ ] 电力供应（双路+UPS）、电磁防护
- [ ] 取得物理环境等保测评结论（或复用政务云测评）

---

## 三、安全通信网络

| # | 检查项 | 平台落实 | 验证方式 |
|---|---|---|---|
| 1 | 网络架构冗余（核心双机热备） | K8s 多节点 + 中间件集群 | 拓扑图 + 故障演练 |
| 2 | 通信传输保密性（TLS 1.3） | Gateway 全链路 TLS，内部 mTLS | 抓包验证/证书检查 |
| 3 | 通信完整性（签名/哈希） | JWT 签名 + 消息摘要 | 抓包校验 |
| 4 | 可信接入（边缘网关认证） | iot-ingest 双向证书 | 网关证书台账 |
| 5 | 带宽与可用性 | QoS 保障应急链路优先 | 压力测试 |

---

## 四、安全区域边界

| # | 检查项 | 平台落实 | 验证方式 |
|---|---|---|---|
| 1 | 边界访问控制（ACL/防火墙） | K8s NetworkPolicy 默认拒绝 + WAF | `kubectl get networkpolicy` |
| 2 | 接入控制（VPN/专线） | 企业端经 VPN/专线接入 | 接入日志 |
| 3 | 入侵防范（WAF + IDS） | WAF 规则 + 黑名单联动 | 告警记录 |
| 4 | 恶意代码防范 | 镜像扫描（Trivy）+ 网关防病毒 | 扫描报告 |
| 5 | 安全审计（边界日志） | Ingress/Nginx 访问日志 → ELK | 日志留存 ≥6 月 |
| 6 | 边界恶意流量隔离 | Sentinel 限流熔断 | 压测报告 |

---

## 五、安全计算环境

### 5.1 身份鉴别

- [ ] **口令复杂度**：密码策略（长度≥8、大小写+数字+特殊字符、定期更换）经 Nacos 配置下发
- [ ] **双因子认证**：admin/应急指挥角色启用 MFA（TOTP/短信）
- [ ] **登录失败处理**：连续 5 次锁定，经 iam-svc 实现
- [ ] **口令加密存储**：bcrypt/Argon2，禁止明文/弱哈希
- [ ] **定期鉴别**：90 天强制改密提醒

### 5.2 访问控制

- [ ] **三级 RBAC**：模块级/功能级/数据级（行级 tenant/enterprise 隔离）
- [ ] **默认拒绝**：无权限返回 401/403，最小权限分配
- [ ] **权限分离**：系统管理员 ≠ 安全审计员 ≠ 操作员（三员分立）
- [ ] **越权防护**：服务端校验数据级权限（不依赖前端）

### 5.3 安全审计

- [ ] **全覆盖**：操作日志（人/时间/IP/内容）只追加（WORM）
- [ ] **不可删除**：审计日志独立存储，普通管理员不可删
- [ ] **留存期**：≥ 6 个月（等保要求），关键审计 ≥ 1 年
- [ ] **集中审计**：ELK 统一收集，异常操作告警

### 5.4 入侵防范

- [ ] **最小化服务**：容器仅含必要进程，`readOnlyRootFilesystem`
- [ ] **漏洞修复**：基础镜像 CVE 扫描，及时补丁
- [ ] **多余账户清理**：镜像无默认弱口令账户
- [ ] **关键命令监控**：kubectl exec 审计

### 5.5 恶意代码防范

- [ ] 上传文件（隐患随手拍/预案附件）落 MinIO，类型白名单 + 病毒扫描
- [ ] 文件服务不执行上传内容

### 5.6 数据完整性 / 保密性

- [ ] **存储加密**：敏感字段 AES-256（证照、位置、视频元数据）；数据库 TDE
- [ ] **传输加密**：全链路 TLS 1.3
- [ ] **完整性校验**：配置/主数据变更留痕，防篡改
- [ ] **备份恢复**：日全量 + 实时增量（WAL），恢复演练 RTO≤30min/RPO≤5min

### 5.7 个人信息保护（移动端）

- [ ] 人员定位/人脸数据最小化采集、脱敏展示
- [ ] 用户授权同意机制（App 隐私政策）

## 六、安全管理中心

| # | 检查项 | 平台落实 |
|---|---|---|
| 1 | 系统管理（配置/审计员分立） | Nacos 配置权限 + iam 三员角色 |
| 2 | 审计管理（集中收集+告警） | ELK + 审计 Topic `audit.log` |
| 3 | 安全管理（策略统一下发） | Sentinel 规则 + Nacos 配置中心 |
| 4 | 集中监控 | Prometheus + Grafana（服务/中间件/安全指标） |
| 5 | 统一告警 | 告警引擎 → notification（短信/邮件/大屏） |

## 七、安全管理制度/机构/人员/建设/运维

| 类别 | 关键项 | 交付物 |
|---|---|---|
| 管理制度 | 安全方针、管理办法、操作规程 | 《信息安全管理制度汇编》 |
| 管理机构 | 设立安全领导小组、三员岗位 | 组织架构文件 |
| 管理人员 | 背景审查、培训、考核 | 培训记录 |
| 建设管理 | 定级备案、测评、外包管控 | 等保备案证明、测评报告 |
| 运维管理 | 变更/补丁/备份/演练/事件处置 | 运维台账、应急演练记录 |

## 八、平台专项检查表（逐项打勾）

### 8.1 认证与授权

- [ ] OAuth2.1 + OIDC 接入，token 有效期 ≤ 2h + 刷新
- [ ] 移动端 PUSH 令牌绑定设备，注销即失效
- [ ] 数据级权限按 `X-Tenant-Id` + `enterpriseId` 行级过滤（JDBC Filter）
- [ ] 应急指挥角色独立授权，避免越权调度

### 8.2 传输与存储

- [ ] 外部 HTTPS/TLS 1.3；内部服务 mTLS 开启
- [ ] 敏感字段 AES-256；数据库 TDE 开启
- [ ] 备份加密存储异地副本

### 8.3 审计与日志

- [ ] 所有写操作入 `audit.log`（WORM）
- [ ] 操作日志含操作人/IP/时间/内容，不可删
- [ ] 日志集中 ELK，留存 ≥ 6 月

### 8.4 网络安全

- [ ] K8s NetworkPolicy 默认拒绝，仅放行 Gateway 入向 + 中间件出向
- [ ] WAF 启用并规则更新
- [ ] Sentinel 限流防刷/防雪崩

### 8.5 应用安全

- [ ] 输入校验/防注入（MyBatis-Plus 预编译）
- [ ] 文件上传白名单 + 扫描
- [ ] 接口防重放（X-Request-Id + 时间戳校验）
- [ ] 容器非 root 运行 + 只读根文件系统

### 8.6 数据安全与应急

- [ ] 主数据强一致（Saga），IoT 最终一致
- [ ] 应急链路降级兜底（emergency-svc 不依赖外部可用性）
- [ ] 年度/季度应急演练记录（含数据恢复）

### 8.7 供应链与镜像

- [ ] 私有镜像仓库 + 镜像签名
- [ ] CI 阶段 Trivy 漏洞扫描阻断高危
- [ ] 基础镜像来源可信、最小化

---

## 九、测评配合建议

1. **资产梳理**：导出 14 个微服务 + 中间件资产清单（名称/版本/责任人）。
2. **拓扑图**：提供网络拓扑与安全区域划分图（对应第四、五章）。
3. **策略导出**：Nacos 配置、NetworkPolicy、RBAC 角色导出备查。
4. **日志抽查**：准备 1 周审计日志样本，验证不可删与完整性。
5. **渗透测试**：上线前第三方渗透（重点关注 Gateway、文件上传、越权）。
6. **整改闭环**：对测评差异项建立台账，限期整改并复测。

---

# 七、CI-CD GitLab Pipeline 示例

---

> **版本**：V1.0 | **日期**：2026-08-22 **配套文档**：《化工园区信息化平台-微服务架构设计方案.md》《化工园区信息化平台-Kubernetes-Helm部署清单示例.md》 **目标**：14 个微服务 + 中间件从代码提交到 K8s 私有化集群的自动化构建、扫描、部署、验证。

---

## 目录


1. 流水线总体设计
2. 仓库与分支策略
3. 全局 .gitlab-ci.yml
4. 构建阶段（Maven + 镜像）
5. 安全扫描阶段
6. 部署阶段（Helm 到 K8s）
7. 验证与回滚
8. 关键变量与 Secrets


---

## 一、流水线总体设计

```
代码提交 ──▶ [build] ──▶ [test] ──▶ [scan] ──▶ [package] ──▶ [deploy:dev]
                                                                    |
                                                  [deploy:stag] ─▶ [e2e] ─▶ [deploy:prod
                                                                                  |
                                                                        [smoke] ─▶ [rollbac
```

| 阶段 | 动作 | 耗时目标 |
|---|---|---|
| build | Maven 编译 + 单元测试 | ≤ 5 min |
| test | 集成测试 + 契约测试 | ≤ 10 min |
| scan | Trivy 镜像 + SonarQube + 依赖审计 | ≤ 8 min |
| package | 构建镜像推私有仓库 | ≤ 5 min |
| deploy | Helm upgrade（按环境） | ≤ 5 min |
| verify | 健康检查 + 冒烟 | ≤ 3 min |

---

## 二、仓库与分支策略

- **单仓多模块（monorepo）**：`platform/` 下 `services/*`、`middleware/`、`charts/`

- **分支**：`main`（生产）、`release/*`（预发）、`develop`（集成）、`feature/*`
- **环境映射**：`develop→dev`、`release/*→stag`、`main→prod（手动）`
- **标签**：`v1.x.x` 触发生产发布

---

## 三、全局 .gitlab-ci.yml

```
stages:
  - build
  - test
  - scan
  - package
  - deploy
  - verify

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2 -B -Dorg.slf4j.simpleLogger.log.org.apache.maven.cli.transfe
  REGISTRY: "registry.chem-park.local"
  CHART_DIR: "charts/chem-park-platform"
  NAMESPACE: "chem-park-app"

include:
  - local: '/ci/build.yml'
  - local: '/ci/scan.yml'
  - local: '/ci/deploy.yml'

cache:
  paths:
    - .m2/repository
    - node_modules
```

---

## 四、构建阶段（Maven + 镜像）

```
# ci/build.yml
.build:
  stage: build
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn -q clean compile
  rules:
```

```
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
    - if: '$CI_COMMIT_BRANCH'

unit-test:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn -q test
    - mvn jacoco:report
  coverage: '/Total.*?([0-9]{1,3})%/'
  artifacts:
    reports:
      junit: '**/target/surefire-reports/TEST-*.xml'
    paths:
      - '**/target/site/jacoco/'

contract-test:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn -q -Pcontract-test verify   # Spring Cloud Contract 校验接口契约
  rules:
    - if: '$CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH =~ /^release/'

docker-build:
  stage: package
  image: docker:24
  services: [docker:24-dind]
  script:
    - |
      SERVICE=$SERVICE_NAME
      TAG=${CI_COMMIT_SHORT_SHA}
      docker build -t $REGISTRY/$SERVICE:$TAG -f services/$SERVICE/Dockerfile .
      docker push $REGISTRY/$SERVICE:$TAG
      echo "IMAGE=$REGISTRY/$SERVICE:$TAG" >> build.env
  artifacts:
    reports:
      dotenv: build.env
  parallel:
    matrix:
      - SERVICE_NAME: [iam-svc, mdm-svc, iot-ingest-svc, device-svc, safety-svc,
                       monitor-svc, closedmgmt-svc, env-svc, energy-svc, corridor-svc,
                       emergency-svc, operation-svc, ioc-svc, gateway-svc]
```

> 每个微服务 `Dockerfile` 基于 `eclipse-temurin:17-jre` 最小化镜像，非 root 运行。

---

## 五、安全扫描阶段

```
# ci/scan.yml
trivy-image:
  stage: scan
  image: aquasec/trivy:0.50
  script:
    - trivy image --exit-code 1 --severity CRITICAL,HIGH $IMAGE
    - trivy image --format sarif --output trivy.sarif $IMAGE
  artifacts:
    reports: { sast: trivy.sarif }
  needs: [docker-build]

sonarqube:
  stage: scan
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn sonar:sonar -Dsonar.host.url=$SONAR_URL -Dsonar.token=$SONAR_TOKEN
  rules:
    - if: '$CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH == "develop"'

dependency-audit:
  stage: scan
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn org.owasp:dependency-check-maven:check   # CVE 依赖审计
  allow_failure: true
```

> `trivy` 阻断 CRITICAL/HIGH；`dependency-check` 失败不阻塞但告警（可配置为阻断）。

---

## 六、部署阶段（Helm 到 K8s）

```
# ci/deploy.yml
.deploy:
  stage: deploy
```

```
  image: bitnami/kubectl:1.29
  before_script:
    - kubectl config use-context $K8S_CONTEXT
  script:
    - |
      helm dependency build $CHART_DIR
      helm upgrade --install chem-park $CHART_DIR \
        -n $NAMESPACE --create-namespace \
        -f $CHART_DIR/values.yaml -f $CHART_DIR/values-$ENV.yaml \
        --set "services.$SERVICE.tag=$TAG" \
        --wait --timeout 300s

deploy-dev:
  extends: .deploy
  variables: { ENV: dev }
  rules:
    - if: '$CI_COMMIT_BRANCH == "develop"'

deploy-stag:
  extends: .deploy
  variables: { ENV: stag }
  rules:
    - if: '$CI_COMMIT_BRANCH =~ /^release/'

deploy-prod:
  extends: .deploy
  variables: { ENV: prod }
  when: manual          # 生产手动触发
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/'
```

> 灰度：生产先 `--set monitor.replicas=3` 滚动，配合 Gateway 权重路由小流量验证。

---

## 七、验证与回滚

```
smoke-test:
  stage: verify
  image: curlimages/curl
  script:
    - curl -fsS https://$DOMAIN/api/ioc/overview -H "Authorization: Bearer $TEST_TOKEN" | grep -q
```

```
    - curl -fsS https://$DOMAIN/actuator/health | grep -q UP
  rules:
    - if: '$CI_COMMIT_TAG'

rollback:
  stage: verify
  when: on_failure
  image: bitnami/kubectl:1.29
  script:
    - helm rollback chem-park -n $NAMESPACE
    - kubectl rollout status deploy/$SERVICE -n $NAMESPACE
  rules:
    - if: '$CI COMMIT TAG'
```

> 监控联动：部署后 Prometheus 检查 5xx 率/延迟，超阈值自动 `helm rollback`。

---

## 八、关键变量与 Secrets

| 变量 | 作用 | 存储 |
| --- | --- | --- |
| `REGISTRY` | 私有镜像仓库地址 | CI Variable |
| `K8S_CONTEXT` | 集群 kubeconfig 上下文 | CI File Variable |
| `SONAR_TOKEN` | SonarQube 令牌 | masked secret |
| `DB_PASSWORD` / `TLS_KEY` | 数据库/证书 | Kubernetes Secret（由 Vault 注入） |
| `TEST_TOKEN` | 冒烟测试 token | masked secret |

> Secrets 经 GitLab CI Masked + Vault 注入，禁止明文入库；kubeconfig 使用最小权限 SA。

---

# 八、应急演练与容灾切换预案

> **版本**：V1.0 | **日期**：2026-08-22 **配套文档**：《化工园区信息化平台-微服务架构设计方案.md》《化工园区信息化平台-Kubernetes-Helm部署清单示例.md》 **目标**：满足 BRD"系统可用率 99.9%、故障恢复 ≤30min、应急调度 ≤5min"及等保三级要求；规范日常演练与故障切换流程。

---

# 目录


1. 总则与目标
2. 角色与职责（应急组织）
3. 故障等级定义（RTO/RPO）
4. 容灾架构基础
5. 应急演练方案（桌面+实战）
6. 容灾切换流程（按场景）
7. 回切与复盘
8. 演练记录与测评配合


## 一、总则与目标

- **方针**：平急结合、预防为主、快速恢复、最小影响。
- **可用性目标**：年意外宕机 ≤ 3 次，操作失败率 < 0.5%，RTO ≤ 30min，RPO ≤ 5min。
- **演练频次**：每半年综合实战演练 1 次 + 每季度专项桌面推演（应急调度/数据恢复/网络安全）。

## 二、角色与职责（应急组织）

| 角色 | 职责 |
| --- | --- |
| 应急总指挥（管委会分管安全副主任） | 启动/终止响应，决策切换 |
| 平台技术组长（信息化科室） | 故障定位、指挥切换执行 |
| 运维组（K8s/中间件） | 执行 Pod 恢复、Helm 回滚、备份恢复 |
| 安全组（iam/审计） | 安全事件研判、账号冻结 |
| 业务组（安全科/环保科/应急指挥中心） | 业务降级确认、手工兜底 |
| 厂商支撑 | 远程协助、组件级修复 |

## 三、故障等级定义（RTO/RPO）

| 等级 | 定义 | 示例 | RTO | RPO |
| --- | --- | --- | --- | --- |
| P1 致命 | 核心业务全停/数据丢失风险 | 所有服务不可用、DB 损坏 | ≤30min | ≤5min |
| P2 严重 | 单核心服务不可用 | monitor/emergency 宕机 | ≤30min | ≤15min |
| P3 一般 | 非核心服务/部分功能降级 | operation/OA 异常 | ≤2h | ≤1h |
| P4 轻微 | 性能劣化/单副本重启 | 某 Pod OOM 自愈 | ≤4h | — |

## 四、容灾架构基础

- **多副本**：14 服务 ≥2 副本；monitor/emergency/ioc 高优先级 + 反亲和。
- **中间件集群**：Nacos(3)/Kafka(3,ISR=2)/PostgreSQL流复制/TimescaleDB/Redis哨兵/MinIO(EC)。
- **备份**：PG/TimescaleDB 日全量 + WAL 实时增量（RPO≤5min）；备份加密异地。
- **边缘容灾**：iot-ingest 边缘网关断网本地缓存，恢复补传（采集不丢）。
- **应急降级**：emergency-svc 本地预案基线，外部服务不可用时仍可调度。

---

## 五、应急演练方案（桌面+实战）

### 5.1 桌面推演（季度）

- **课题**：重大危险源报警→应急联动→资源调度全链路（验证 AlarmEvent→emergency→notification）。
- **步骤**：触发模拟告警 → 验证大屏切换 ≤3s → 验证多通道通知 → 验证资源推荐 ≤3km。
- **输出**：《桌面推演记录表》+ 问题台账。

### 5.2 实战演练（半年）

- **课题 A 数据恢复**：模拟 TimescaleDB 主库失效，演练从备份恢复 + 校验点位连续性。
- **课题 B 服务级故障**：kill `monitor-svc` 全部 Pod，验证 HPA 重建 + 大屏降级展示。
- **课题 C 网络分区**：隔离 `chem-park-middleware`，验证边缘缓存 + 业务降级。
- **课题 D 安全事件**：模拟越权访问，验证审计告警 + 账号冻结。

---

## 六、容灾切换流程（按场景）

### 6.1 场景一：单微服务故障（P2/P3）

```
1. 告警触发（Prometheus）→ 运维组确认
2. kubectl get pods 确认 CrashLoop / OOM
3. 自动：HPA/重启自愈；超时则：
   helm rollback chem-park -n chem-park-app --set services.<svc>.tag=<last-good>
4. 验证：/actuator/health + 冒烟接口
5. 业务组确认功能恢复，关闭事件
```

### 6.2 场景二：数据库主库失效（P1）

```
1. 总指挥启动 P1，技术组长定位（PG 流复制延迟/主库宕机）
2. 运维组提升备库为主（Patroni/流复制 failover）
3. 应用连接串经 Nacos 动态切换（无需重启）
```

4. 校验：最近 5min WAL 已应用（RPO 核查）
5. 若主库数据损坏：从最近全量 + WAL 增量恢复至临时实例，校验后切换
6. 业务组全量验证（安全/环保/应急核心链路）

### 6.3 场景三：K8s 集群级故障（P1）

1. 总指挥决策切换至同城灾备集群（备用 K8s）
2. 运维组：备集群 helm upgrade 拉起全套（复用镜像仓库）
3. 恢复最新备份至备集群 DB（RPO 核查）
4. DNS/Ingress 切换至备集群入口（或 Gateway 权重切流）
5. 边缘网关重连备集群，积压数据补传
6. 大屏/移动端验证，业务组确认

### 6.4 场景四：安全事件（越权/入侵）

1. 安全组研判，冻结可疑账号（iam-svc 强制下线）
2. 网络组收紧 NetworkPolicy，隔离异常 Pod
3. 审计组导出 audit.log 留存证据（WORM）
4. 修复漏洞 → CI/CD 重发镜像 → 滚动替换
5. 复盘并补强规则（Sentinel/WAF）

---

## 七、回切与复盘

- **回切条件**：原环境修复 + 数据校验一致 + 总指挥批准。
- **回切步骤**：备→主反向同步校验 → 流量切回 → 观察 24h。
- **复盘**：演练/故障后 3 个工作日内出《复盘报告》（时间线/根因/改进项/责任人/时限）。
- **改进闭环**：差异项入台账，关联 CI/CD 与架构优化。

---

## 八、演练记录与测评配合

| 记录项 | 说明 |
| --- | --- |
| 演练计划/方案 | 每半年/每季度 |
| 演练签到与角色 | 证明组织到位 |
| 时间线记录 | 故障发现→切换→恢复各节点耗时（对比 RTO） |
| RPO 核查结果 | 数据丢失量 |
| 问题台账与整改 | 关联等保测评 |

| 记录项 | 说明 |
| --- | --- |
| 复盘报告 | 归档 ≥ 3 年 |

> 演练记录与等保"运维管理-应急响应"条款直接对应，作为测评证据。

---

# 九、架构决策记录（ADR）

> **版本**：V1.0 | **日期**：2026-08-22 **配套文档**：本目录下全部架构方案文档 **格式**：遵循 MADR（Markdown Architecture Decision Records）轻量格式 **说明**：每条 ADR 记录一项关键架构决策，包含状态、背景、决策选项、最终决策与后果。新增/变更决策追加记录，不修改历史条目（仅置状态为 Superseded）。

---

## 目录


- ADR-001 微服务拆分边界（DDD 业务域）
- ADR-002 服务间通信：同步 Feign + 异步 Kafka 双通道
- ADR-003 分布式事务采用 Seata Saga
- ADR-004 IOC 大屏采用 CQRS 只读聚合
- ADR-005 时序数据采用 TimescaleDB
- ADR-006 注册中心与配置中心统一采用 Nacos
- ADR-007 应急指挥服务独立高可用隔离域
- ADR-008 安全合规遵循等保 2.0 三级内建
- ADR-009 API 网关统一鉴权与限流
- ADR-010 容器化部署采用 Kubernetes + Helm
- ADR-011 多租户隔离策略（Schema / 库 + 行级）
- ADR-012 部署与交付分期对齐产品路线图


---

## ADR-001 微服务拆分边界（DDD 业务域）

- **状态**：已采纳（Accepted）
- **背景**：原型含 10 模块 46 界面，业务跨度大（安全/环保/应急/能碳/管廊/运营）。单体架构无法满足"分期交付、独立扩容、故障隔离"，且高频监测会拖垮台账类业务。
- **决策选项**： 1. 单体应用（快速但耦合高，否决） 2. 按技术层拆分（gateway/dao/service，否决：业务仍耦合） 3. **按业务域（DDD）拆分为 14 个服务**（采纳）

- **最终决策**：按 DDD 业务域拆分为 14 个微服务，每服务独立库、单一职责。将 monitor 与 device、emergency 独立隔离。
- **后果**：
- 优点：职责清晰、可独立部署扩容、故障隔离、对齐分期交付。
- 代价：分布式事务/运维复杂度上升（由 ADR-003/010 缓解）。

---

## ADR-002 服务间通信：同步 Feign + 异步 Kafka 双通道

- **状态**：已采纳（Accepted）
- **背景**：页面查询需低延迟响应，跨域事件（告警/应急）需解耦削峰且不可丢。
- **决策选项**： 1. 全同步 REST（简单，但耦合高、无削峰，否决） 2. 全异步消息（解耦，但实时查询体验差，否决） 3. **同步 Feign（查询）+ 异步 Kafka（事件/数据流）双通道**（采纳）
- **最终决策**：实时请求走 OpenFeign（经 Gateway，带熔断）；告警/预警/应急/数据流走 Kafka（保序、不丢、可重放）。
- **后果**：
- 优点：兼顾实时性与解耦；Kafka 分区键保序，至少一次投递。
- 代价：需维护两套通信语义；消费者幂等（eventId）必须实现。

---

## ADR-003 分布式事务采用 Seata Saga

- **状态**：已采纳（Accepted）
- **背景**：作业票审批、隐患整改等跨多服务长流程，需最终一致且可补偿（驳回/退回修改）。2PC 会长期持锁拖垮高频监测库。
- **决策选项**： 1. 2PC/XA（强一致但持锁久，否决） 2. TCC（性能好但侵入强，适合短资源锁定） 3. **Saga（长流程补偿，采纳）；TCC 仅用于应急资源锁定**
- **最终决策**：跨服务长流程用 Seata Saga（前向+补偿），状态机守卫单服务状态跃迁；应急资源锁定用 TCC。
- **后果**：
- 优点：最终一致、不长期持锁、业务可驳回回滚。
- 代价：补偿逻辑需幂等设计；需 Saga 可视化与死信运维。

---

## ADR-004 IOC 大屏采用 CQRS 只读聚合

- **状态**：已采纳（Accepted）
- **背景**：IOC 综合态势一张图需聚合安全/环保/应急/能源/物流五域数据，30s 刷新、≤5s 加载，且要"平急结合"秒级切换。
- **决策选项**： 1. 大屏直连各业务库（耦合高、影响业务库性能，否决） 2. **独立 ioc-svc 只读聚合（CQRS 读侧）：订阅事件 + 主动拉取组装**（采纳）
- **最终决策**：`ioc-svc` 不持有业务真相源，仅订阅 Kafka 告警事件 + Feign 拉取各域摘要写入本地聚合缓存，大屏只读该服务。

- **后果**：
- 优点：业务库零压力、大屏高性能、切换快。
- 代价：聚合数据有秒级延迟；需处理缓存失效与降级兜底。

---

## ADR-005 时序数据采用 TimescaleDB

- **状态**：已采纳（Accepted）
- **背景**：IoT 点位、重大危险源参数、环保监测等高频写入（10s 刷新）且需长期历史回溯、趋势统计。
- **决策选项**： 1. 纯关系表（写入/查询性能差，否决） 2. 纯时序库（如 InfluxDB，但与 PostgreSQL 生态割裂、多租户复杂，否决） 3. **TimescaleDB（PostgreSQL 扩展，兼容 SQL + 超表 + 连续聚合）**（采纳）
- **最终决策**：业务关系数据用 PostgreSQL，高频时序数据用 TimescaleDB 超表 + 连续聚合 + 压缩降采样。
- **后果**：
- 优点：SQL 统一、运维简单、聚合视图减负、压缩比高。
- 代价：需独立管理超表保留/压缩策略；与业务库备份需协调。

---

## ADR-006 注册中心与配置中心统一采用 Nacos

- **状态**：已采纳（Accepted）
- **背景**：需服务发现、动态配置、多环境隔离，并尽量降低组件数量。
- **决策选项**： 1. Eureka + Config + Bus（组件多，否决） 2. Consul（功能全但团队熟悉度低，备选） 3. **Nacos（注册+配置一体，国产、云原生友好）**（采纳）
- **最终决策**：统一采用 Nacos 作为注册中心与配置中心（Namespace 环境隔离、Group 服务隔离、敏感配置加密）。
- **后果**：
- 优点：组件收敛、动态刷新、国产合规。
- 代价：Nacos 自身需集群高可用（≥3 节点）；注册中心抖动依赖本地缓存降级。

---

## ADR-007 应急指挥服务独立高可用隔离域

- **状态**：已采纳（Accepted）
- **背景**：应急是"平急结合"指挥中枢，事故响应"黄金十分钟"要求调度不被日常流量/外部依赖拖垮。
- **决策选项**： 1. 应急作为普通业务服务（故障会阻断调度，否决） 2. **emergency-svc 独立部署单元 + 高优先级 + 降级兜底 + 本地预案基线**（采纳）
- **最终决策**：`emergency-svc` 独立资源配额（priorityClass high-priority，多副本，反亲和）；对其他域调用均带熔断降级，本地缓存预案/资源基线。
- **后果**：
- 优点：应急链路韧性最强，外部不可用仍可调度。
- 代价：需维护基线数据一致性；资源成本略高。

## ADR-008 安全合规遵循等保 2.0 三级内建

- **状态**：已采纳（Accepted）
- **背景**：BRD 明确要求等保 2.0 三级；化工园区属关键信息基础设施相关场景。
- **决策选项**： 1. 上线后补安全（测评风险高，否决） 2. **安全左移、内建于架构各层**（采纳）
- **最终决策**：传输 TLS 1.3、存储 AES-256/TDE、三级 RBAC + 行级多租户、审计 WORM、网络隔离（NetworkPolicy）、三员分立。
- **后果**：
- 优点：测评可顺利通过，安全可验证。
- 代价：部分性能开销（加密/审计）；需持续合规运营。

## ADR-009 API 网关统一鉴权与限流

- **状态**：已采纳（Accepted）
- **背景**：多端（Web/大屏/移动/企业/监管）接入，需统一入口做认证、鉴权、限流、灰度。
- **决策选项**： 1. 各服务自理鉴权（重复、易漏，否决） 2. **Spring Cloud Gateway 统一鉴权 + Sentinel 限流 + 灰度**（采纳）
- **最终决策**：所有外部请求经 Gateway，校验 OAuth2.1 token、解析三级权限、按服务维度限流；紧急态 emergency 路由最高优先级。
- **后果**：
- 优点：安全边界清晰、限流防雪崩、灰度可控。
- 代价：网关成为关键路径，需高可用（多副本 + 熔断）。

## ADR-010 容器化部署采用 Kubernetes + Helm

- **状态**：已采纳（Accepted）
- **背景**：14 服务 + 中间件需标准化交付到私有化/政务云集群，支持滚动升级、弹性伸缩、故障自愈。
- **决策选项**： 1. 物理机/虚拟机手工部署（慢、易错，否决） 2. **Kubernetes + Helm 标准化部署**（采纳）
- **最终决策**：全部容器化，K8s 编排，Helm Chart 一键部署；HPA 自动扩缩、Pod 反亲和、NetworkPolicy 隔离。
- **后果**：
- 优点：交付快、可复制（多园区 SaaS）、自愈能力强。
- 代价：集群运维门槛；需镜像仓库与 CI/CD 配套（见 CI/CD 文档）。

## ADR-011 多租户隔离策略（Schema / 库 + 行级）

- **状态**：已采纳（Accepted）

- **背景**：平台既可能单园区私有化，也可能多园区 SaaS；需数据隔离且兼顾成本。
- **决策选项**： 1. 共享表无隔离（安全隐患，否决） 2. **单园区：Schema 隔离；SaaS 多园区：库/实例隔离 + 行级 tenant_id 过滤**（采纳）
- **最终决策**：按部署模式选择——私有化按 Schema 隔离（每园区/企业独立 Schema）；SaaS 按实例 + 行级 JDBC Filter 注入 `tenant_id`。
- **后果**：
- 优点：隔离强度可调，灵活适配两种交付。
- 代价：SaaS 模式需中间件（MyCat/ShardingSphere）或 Filter 统一管控，避免越权。

---

## ADR-012 部署与交付分期对齐产品路线图

- **状态**：已采纳（Accepted）
- **背景**：路线图分三期（M1–M36），模块优先级 P0>P1>P2；需架构支撑分期上线而非一次性交付。
- **决策选项**： 1. 一次性全量交付（风险高、周期长，否决） 2. **服务按优先级分批上线，对齐 Phase 1/2/3**（采纳）
- **最终决策**：Phase1 上线 iam/mdm/iot-ingest/device/safety/monitor/closedmgmt/ioc/gateway + 数据中台基础；Phase2 补 env/energy/emergency 补全/ioc 升级；Phase3 补 operation/corridor/AI。
- **后果**：
- 优点：合规达标先行、风险可控、投资分期。
- 代价：前期服务需预留后期接口与事件契约兼容性（向后兼容演进）。

---

## 附录：决策回顾与演进规则

- 新增决策：追加 `ADR-0xx`，状态置 `Proposed` → 评审后 `Accepted`。
- 变更决策：不删改原文，新增 `Superseded by ADR-0yy` 条目。
- 所有 ADR 与架构方案文档、接口契约、部署清单保持一致；如某文档冲突，以 ADR 记录的决策意图为准并同步修订文档。

---